Experiment-9

Generative Adversarial Networks (GANs)

Aim

To study Generative Adversarial Networks (GANs) by implementing a Deep Convolutional GAN (DCGAN) on the MNIST dataset, introducing adversarial training for image generation, and training it for a few epochs to produce digit-like samples, while monitoring progress through epoch-wise "real vs. generated" image panels and a simple quality indicator.

Theory

Generative Adversarial Networks (GANs), proposed by Goodfellow et al. in 2014, are a class of generative models used for learning the underlying data distribution and producing realistic synthetic samples. GANs serve as an alternative to Variational Autoencoders (VAEs) for modelling latent spaces, particularly in image generation tasks. Their key objective is to generate data that is statistically similar to real samples, making the generated outputs difficult to distinguish from actual data.

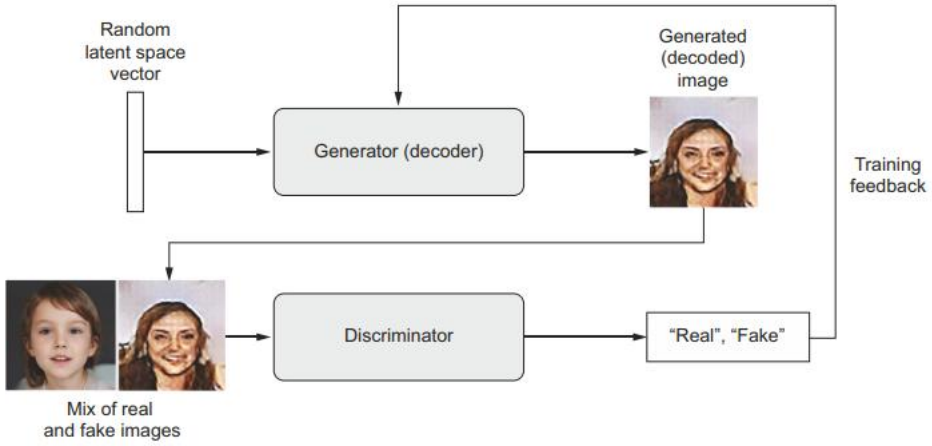


Fig. 1. A generator transforms random latent vectors into images, and a discriminator seeks to tell real images from generated ones. The generator is trained to fool the discriminator.
(Source: F. Chollet, *Deep Learning with Python*, 2nd ed., Manning Publications, 2021.)

A GAN consists of two neural networks trained simultaneously in a competitive framework.

- The generator network takes random noise vectors from a latent space as input and transforms them into synthetic images.
- The discriminator network receives both real images from the training dataset and fake images produced by the generator, and its task is to classify each input as real or generated.

The training process of a GAN can be intuitively understood as a competition between a counterfeit artist and an expert evaluator. Initially, the generator produces poor-quality images, which the discriminator easily identifies as fake. Through continuous feedback from the discriminator, the generator gradually improves its ability to create more realistic images. At the same time, the discriminator becomes more

skilled at detecting subtle differences between real and generated samples. Over successive training iterations, both networks improve, eventually reaching a point where the discriminator finds it difficult to distinguish between real and synthetic images.

Groundtruth MNIST GAN DCGAN

Fig. 2. Side-by-side illustration of (from left-to-right) the MNIST dataset, generations from a baseline GAN, and generations from our DCGAN
(Source: Radford, A., Metz, L., & Chintala, S., *"Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks,"* arXiv:1511.06434, 2015.)

Adversarial Learning

The adversarial learning framework is easiest to understand when both the generator and discriminator are implemented using multilayer perceptrons. In this approach, the generator learns the data distribution by first taking random noise as input and transforming it into data-like samples using a neural network. This generator network maps random vectors to the data space and is trained to produce realistic outputs.

Along with the generator, a discriminator network is used, which is also a neural network that takes an input sample and outputs a single value representing the probability that the sample is real (from the training dataset) rather than generated. The discriminator is trained to correctly distinguish between real data and fake data produced by the generator.

During training, both networks are optimized together in an adversarial manner. The discriminator improves its ability to identify real and fake samples, while the generator improves its ability to fool the discriminator by producing more realistic outputs. This interaction can be viewed as a two-player game, where the generator tries to minimize the discriminator's ability to detect fake samples, and the discriminator tries to maximize its classification accuracy.

$$\min_{G} \max_{D} V(D,G) = E_{x\sim p_{data}(x)}[\log D(x)] + E_{z\sim p_z(z)}[\log(1 - D(G(z)))]$$

where:

- x: A real data sample drawn from the training dataset
- $p_{data}(x)$: True data distribution
- z: Random noise vector sampled from a prior distribution
- $p_z(z)$: Prior noise distribution (usually Gaussian or uniform)
- $G(z;\theta_g)$: Generator network that maps noise z to a synthetic data sample
- θ_g: Parameters (weights) of the Generator
- $D(x;\theta_d)$: Discriminator network that outputs the probability that x is real
- θ_d: Parameters (weights) of the Discriminator
- $D(x) \in [0,1]$: Probability that input x is from real data

Meaning of Each Term:

- $E_{x\sim p_{data}(x)}[\log D(x)]$:
  Encourages the discriminator to correctly classify real samples as real.

- $E_{z\sim p_z(z)}[\log(1-D(G(z)))]$:
  Encourages the discriminator to correctly classify generated samples as fake, while the generator tries to minimize this term to fool the discriminator.

Interpretation:

- The Discriminator tries to maximize this value function by improving its ability to distinguish real from fake data.
- The Generator tries to minimize the function by generating samples that the discriminator classifies as real.

Unlike traditional deep learning models that optimize a fixed loss function, GAN training involves a dynamic optimization process. Each update to the generator affects the discriminator's learning objective and vice versa. Instead of converging to a single minimum, the training seeks an equilibrium between the two competing networks. This adversarial nature makes GANs powerful but also challenging to train, often requiring careful selection of network architectures, hyper-parameters, and training strategies.

Once training is complete, the generator can map any point from the latent space to a realistic output image. However, unlike VAEs, GANs do not explicitly enforce continuity or structure in the latent space, which can make interpolation and control more complex.

Merits of Generative Adversarial Networks:

- **High-Quality Data Generation:**
  GANs are capable of generating high-resolution and highly realistic images, videos, and other types of data. The quality of the generated data is often superior to that produced by other generative models.

- **Unsupervised Learning:**
  GANs can learn to generate data without requiring labelled training data. This is particularly useful in situations where labelled data is scarce or expensive to obtain.

- **Data Augmentation:**
  GANs can generate synthetic data to augment existing datasets, which can be beneficial for training machine learning models, especially in scenarios where real data is limited.

Demerits of Generative Adversarial Networks:

- **Training Instability:**
  Training GANs is notoriously difficult and unstable. The process often suffers from issues such as mode collapse, where the generator produces a limited variety of outputs, and vanishing gradients, where the discriminator becomes too strong, hindering the generator's learning.
- **Mode Collapse:**
  Mode collapse is a common problem in GANs where the generator produces a narrow range of outputs, failing to capture the diversity of the data distribution. This can lead to poor generalization and limited applicability.
- **Susceptibility to Adversarial Attacks:**
  GANs, like other neural networks, can be vulnerable to adversarial attacks, where small perturbations to the input data can significantly impact the output, potentially leading to misleading or harmful results.

Code and Result

Block 1: Import Libraries

INPUT

```
import torch

import torch.nn as nn

import torch.optim as optim

from torchvision import datasets, transforms

from torchvision.utils import make_grid

import matplotlib.pyplot as plt

import numpy as np

import copy
```

OUTPUT:

```
✓ Libraries imported successfully
```

Block 2: Set Device

INPUT

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Using device:", device)
```

OUTPUT:

```
Using device: cpu
```

Block 3: Load and Separate Dataset by Digit Class

INPUT

```
# Dataset preparation - separate by digit class

transform = transforms.Compose([

        transforms.Resize(16),

        transforms.ToTensor(),

        transforms.Normalize((0.5,), (0.5,))

])

full_dataset = datasets.MNIST(root="./data", train=True, transform=transform, download=True)

# Separate data by digit class (0-9)

digit_datasets = {}

digit_loaders = {}

for digit in range(10):

        indices = [i for i, (_, label) in enumerate(full_dataset) if label == digit]

        digit_datasets[digit] = torch.utils.data.Subset(full_dataset, indices[:1500])

        digit_loaders[digit] = torch.utils.data.DataLoader(

                digit_datasets[digit],

                batch_size=64,
```

```
        shuffle=True
    )
    print(f"Digit {digit}: {len(digit_datasets[digit])} images")
```

OUTPUT:

Digit 0: 1500 images

Digit 1: 1500 images

Digit 2: 1500 images

Digit 3: 1500 images

Digit 4: 1500 images

Digit 5: 1500 images

Digit 6: 1500 images

Digit 7: 1500 images

Digit 8: 1500 images

Digit 9: 1500 images

Block 4: Define Hyperparameters

INPUT

```
# Hyperparameters
latent_dim = 100
epochs_per_digit = 25
lr = 0.0002
beta1 = 0.5
```

OUTPUT:

✓ Hyperparameters defined successfully

Block 5: Define Generator Architecture

INPUT

```
# Generator

class Generator(nn.Module):

        def __init__(self):

                super().__init__()

                self.net = nn.Sequential(

                        nn.ConvTranspose2d(latent_dim, 256, 4, 1, 0, bias=False),

                        nn.BatchNorm2d(256),

                        nn.ReLU(True),

                        nn.ConvTranspose2d(256, 128, 4, 2, 1, bias=False),

                        nn.BatchNorm2d(128),

                        nn.ReLU(True),

                        nn.ConvTranspose2d(128, 1, 4, 2, 1, bias=False),

                        nn.Tanh()

        )

        def forward(self, z):

                return self.net(z)
```

OUTPUT:

✓ Generator architecture defined successfully

Block 6: Define Discriminator Architecture

INPUT

```
# Discriminator: 16x16 image -> probability real/fake
```

```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 128, 4, 2, 1, bias=False), # 16x16 -> 8x8
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(128, 256, 4, 2, 1, bias=False), # 8x8 -> 4x4
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(256, 1, 4, 1, 0, bias=False), # 4x4 -> 1x1
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x).view(-1, 1)
```

OUTPUT:

✓ Discriminator architecture defined successfully

Block 7: Define Weight Initialization Function

INPUT

```
# Weight initialization
def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
```

```
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)
```

OUTPUT:

✓ Weight initialization function defined

Block 8: Define Loss Criterion

INPUT

```
criterion = nn.BCELoss()
```

OUTPUT:

✓ Loss criterion defined (Binary Cross Entropy)

Block 9: Define Visualization Function

INPUT

```
def show_real_vs_fake(real, fake, digit):
    real_grid = make_grid(real[:8], nrow=4, normalize=True)
    fake_grid = make_grid(fake[:8], nrow=4, normalize=True)
    plt.figure(figsize=(8, 4))
    plt.subplot(1, 2, 1)
    plt.imshow(real_grid.permute(1, 2, 0))
    plt.title(f"Real Images - Digit {digit}")
    plt.axis("off")
    plt.subplot(1, 2, 2)
    plt.imshow(fake_grid.permute(1, 2, 0))
```

```
    plt.title(f"Generated Images - Digit {digit}")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
```

OUTPUT:

✓ Visualization function defined

Block 10: Define Image Quality Metrics Function

INPUT

```
def calculate_image_quality(images):
    images = images.detach().cpu().numpy()
    diversity = np.std(images)
    sharpness_values = []
    for img in images:
        img_2d = img.squeeze()
        grad_y, grad_x = np.gradient(img_2d)
        sharpness_values.append(np.mean(np.abs(grad_x)) + np.mean(np.abs(grad_y)))
    sharpness = np.mean(sharpness_values)
    contrast = np.mean([img.max() - img.min() for img in images])
    return {
        'diversity': diversity,
        'sharpness': sharpness,
        'contrast': contrast
    }
```

OUTPUT:

✓ Image quality metrics function defined

Block 11: Initialize Training Storage

INPUT

```
trained_generators = {}

metrics_history = {digit: {'g_loss': [], 'd_loss': [], 'quality': []} for digit in range(10)}

moving_avg_window = 3
```

OUTPUT:

✓ Training storage initialized for 10 digits

Block 12: Train GANs for All Digits (0-9) Sequentially

INPUT

```
for digit in range(10):

    print(f"\n{'='*70}")

    print(f"TRAINING DIGIT {digit}")

    print(f"{'='*70}\n")

    generator = Generator().to(device)

    discriminator = Discriminator().to(device)

    generator.apply(weights_init)

    discriminator.apply(weights_init)

    optimizer_G = optim.Adam(generator.parameters(), lr=lr, betas=(beta1, 0.999))

    optimizer_D = optim.Adam(discriminator.parameters(), lr=lr, betas=(beta1, 0.999))

    # Cache one real batch for visualization
```

```
fixed_real_batch, _ = next(iter(digit_loaders[digit]))
fixed_real_batch = fixed_real_batch.to(device)
for epoch in range(epochs_per_digit):
        epoch_g_loss = 0
        epoch_d_loss = 0
        num_batches = 0
        for batch_idx, (real_images, _) in enumerate(digit_loaders[digit]):
                batch_size = real_images.size(0)
                real_images = real_images.to(device)
                real_labels = torch.full((batch_size, 1), 0.9, device=device)
                fake_labels = torch.full((batch_size, 1), 0.1, device=device)
                if batch_idx % 2 == 0:
                        optimizer_D.zero_grad()
                        real_output = discriminator(real_images)
                        d_real_loss = criterion(real_output, real_labels)
                        noise = torch.randn(batch_size, latent_dim, 1, 1, device=device)
                        fake_images = generator(noise)
                        fake_output = discriminator(fake_images.detach())
                        d_fake_loss = criterion(fake_output, fake_labels)
                        d_loss = d_real_loss + d_fake_loss
                        d_loss.backward()
                        optimizer_D.step()
                        epoch_d_loss += d_loss.item()
                else:
```

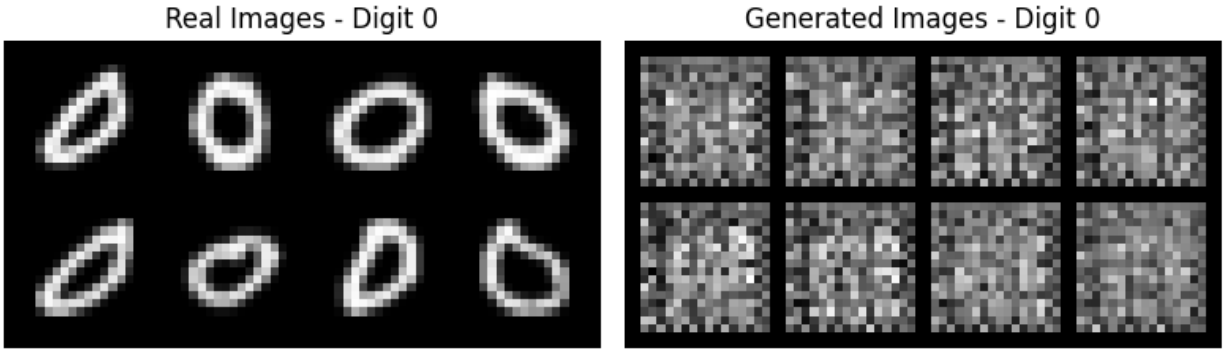
```
                noise = torch.randn(batch_size, latent_dim, 1, 1, device=device)

                fake_images = generator(noise)

            optimizer_G.zero_grad()

            fake_output = discriminator(fake_images)

            g_loss = criterion(fake_output, real_labels)

            g_loss.backward()

            optimizer_G.step()

            epoch_g_loss += g_loss.item()

            num_batches += 1

    avg_g_loss = epoch_g_loss / num_batches

    avg_d_loss = epoch_d_loss / (num_batches // 2)

    with torch.no_grad():

            sample_noise = torch.randn(16, latent_dim, 1, 1, device=device)

            sample_images = generator(sample_noise)

            quality_metrics = calculate_image_quality(sample_images)

    metrics_history[digit]['g_loss'].append(avg_g_loss)

    metrics_history[digit]['d_loss'].append(avg_d_loss)

    metrics_history[digit]['quality'].append(quality_metrics)

    if len(metrics_history[digit]['g_loss']) >= moving_avg_window:

            smoothed_g_loss = sum(metrics_history[digit]['g_loss'][-moving_avg_window:])
            / moving_avg_window

    else:

            smoothed_g_loss = avg_g_loss

    quality = "✅" if smoothed_g_loss < 2.5 else "⚠️" if smoothed_g_loss < 4.5 else "❌"

    if (epoch + 1) % 5 == 0 or epoch == 0:
```

```
                print(f"Epoch [{epoch+1}/{epochs_per_digit}] | "f"G Loss: {avg_g_loss:.3f} | D
                Loss: {avg_d_loss:.3f} | Quality: {quality}")

                # Real vs Generated visualization

                show_real_vs_fake(fixed_real_batch, sample_images, digit)

        trained_generators[digit] = copy.deepcopy(generator.state_dict())

        print(f"\n✓ Digit {digit} training complete!\n")

print("\nALL DIGITS TRAINING COMPLETE!")
```

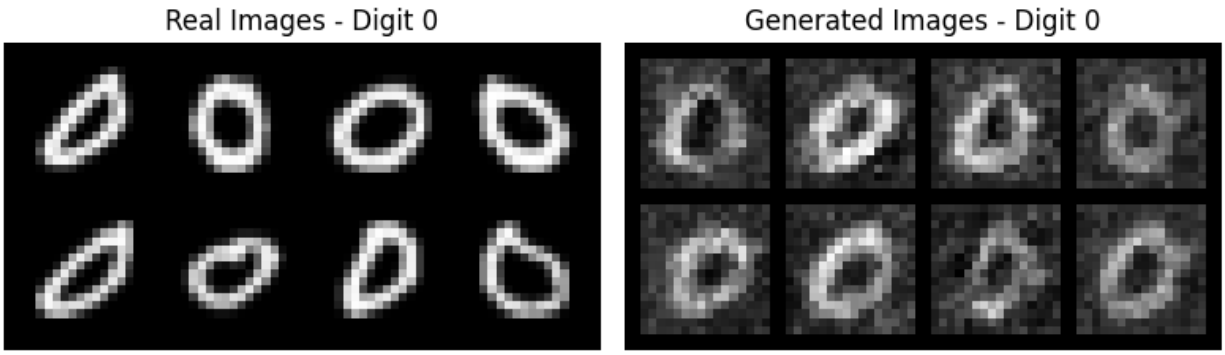
OUTPUT:

```
======================================================================

TRAINING DIGIT 0

======================================================================
```

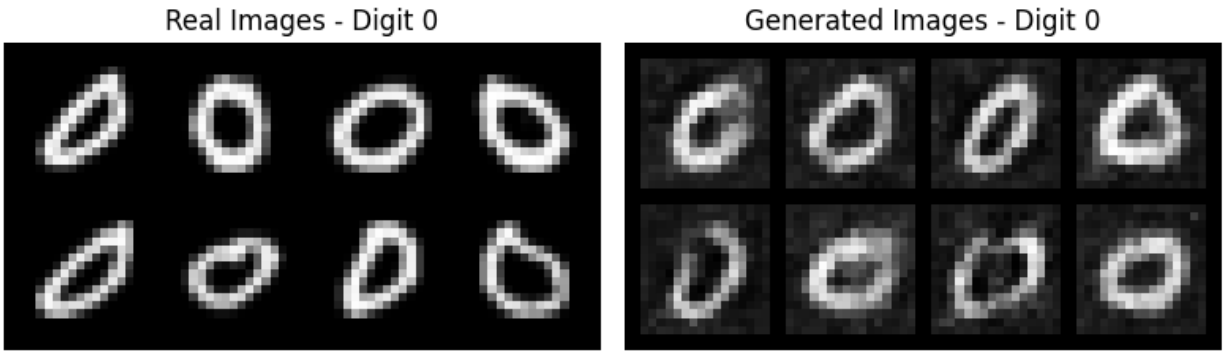
Epoch [1/25] | G Loss: 1.550 | D Loss: 1.324 | Quality: ✓



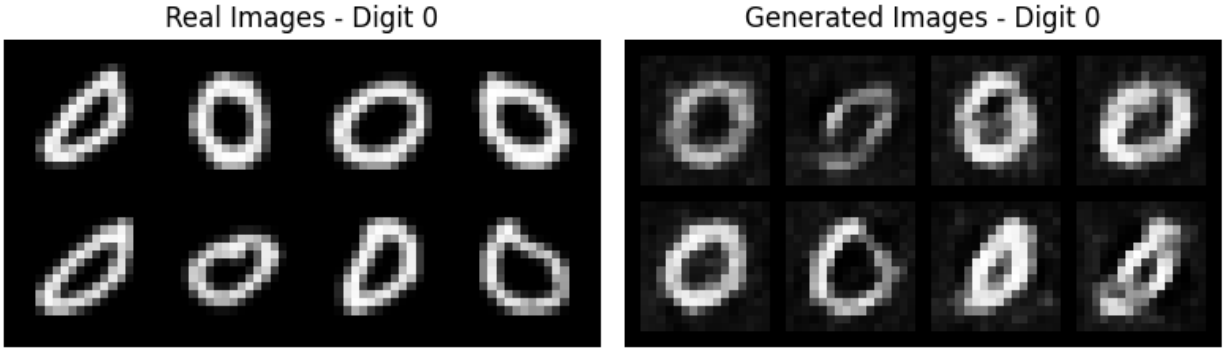
Epoch [5/25] | G Loss: 1.335 | D Loss: 1.113 | Quality: ✓



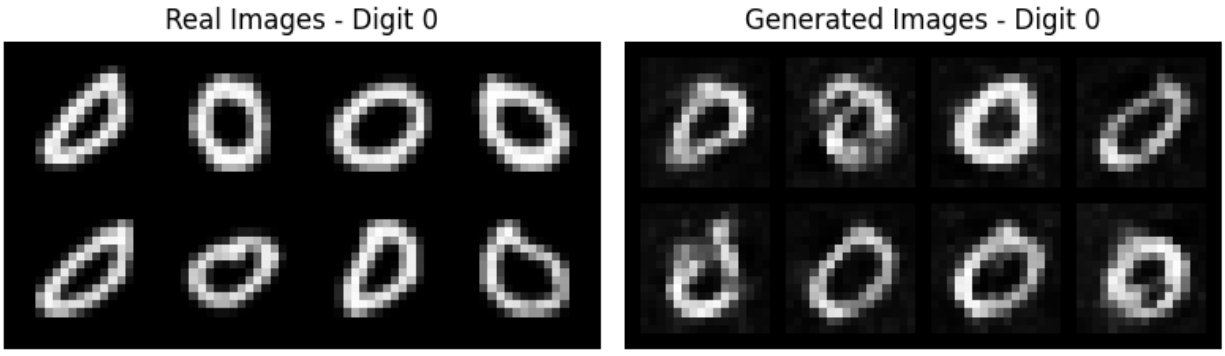
Epoch [10/25] | G Loss: 0.951 | D Loss: 1.250 | Quality: ✓



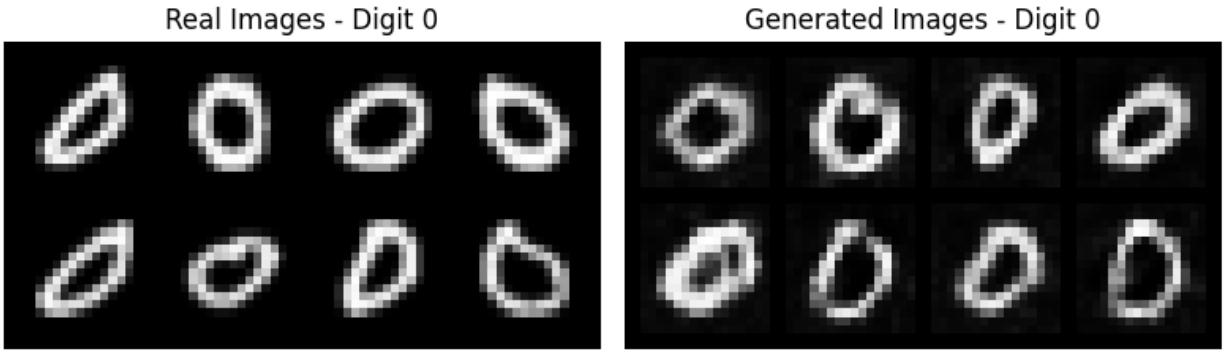
Epoch [15/25] | G Loss: 1.005 | D Loss: 1.247 | Quality: ✓



Epoch [20/25] | G Loss: 0.927 | D Loss: 1.159 | Quality: ✓



Epoch [25/25] | G Loss: 0.938 | D Loss: 1.174 | Quality: ✓



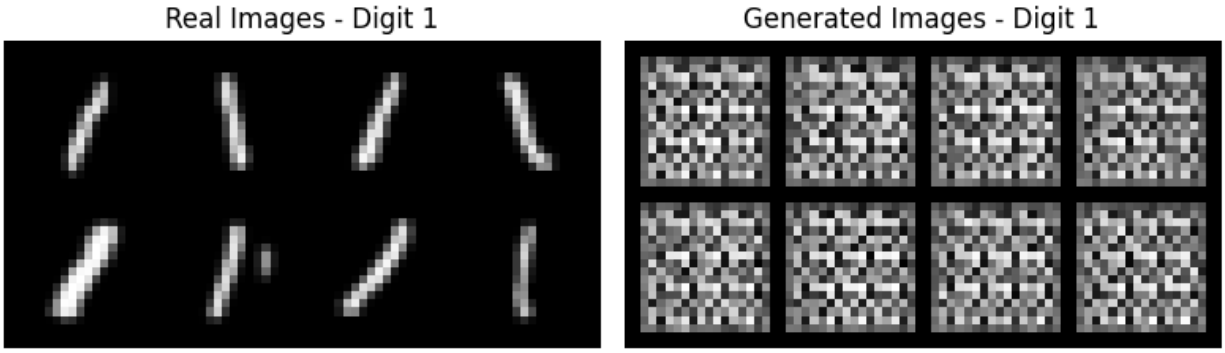
✓ Digit 0 training complete!

==========

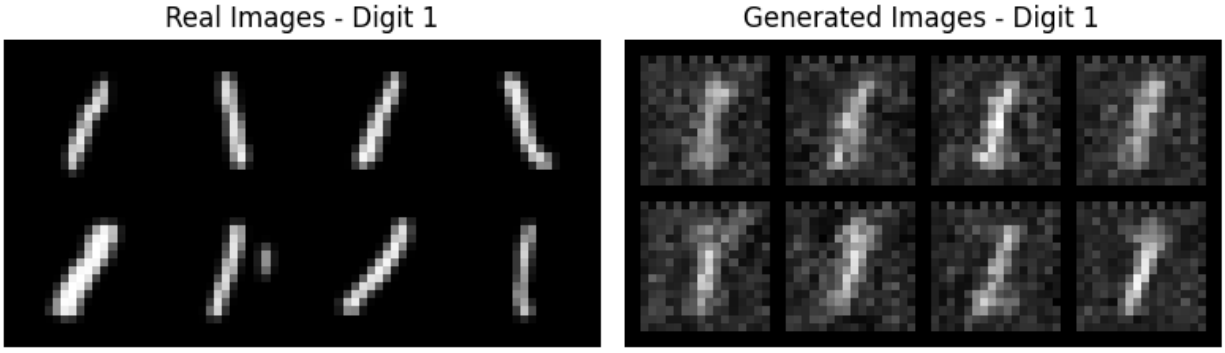
TRAINING DIGIT 1

==========

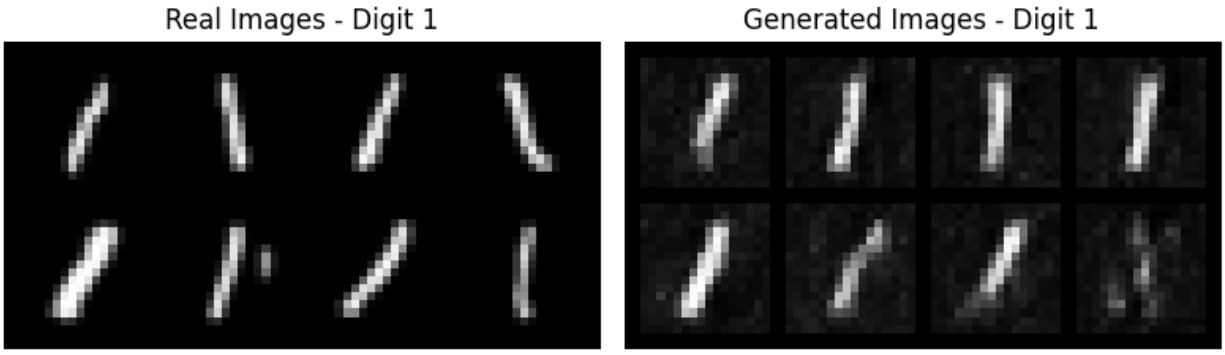
Epoch [1/25] | G Loss: 1.925 | D Loss: 1.238 | Quality: ✓



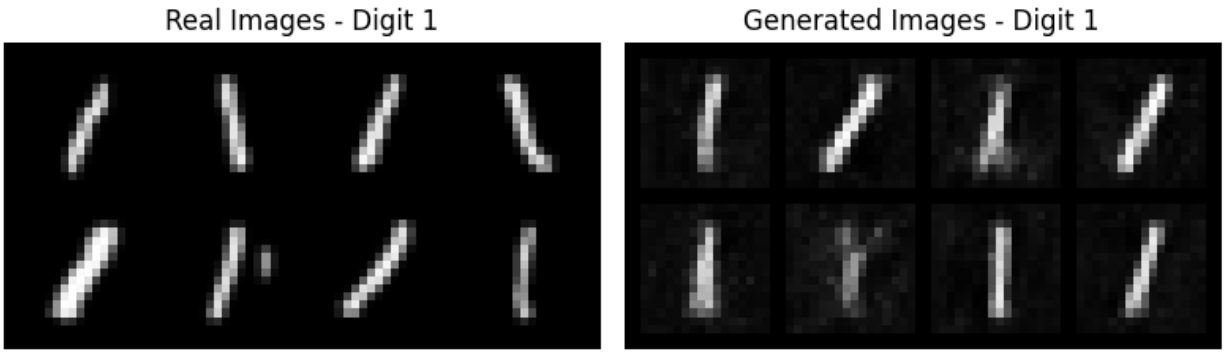
Epoch [5/25] | G Loss: 1.406 | D Loss: 1.027 | Quality: ✓



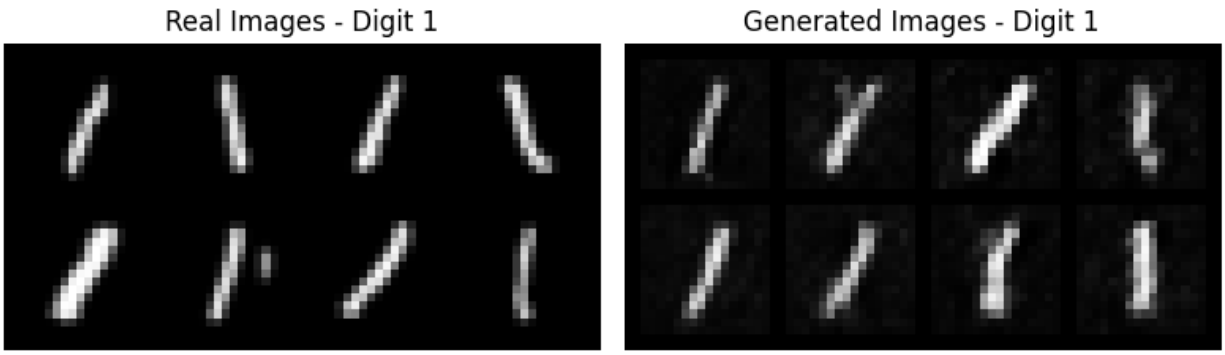
Epoch [10/25] | G Loss: 0.952 | D Loss: 1.213 | Quality: ✓



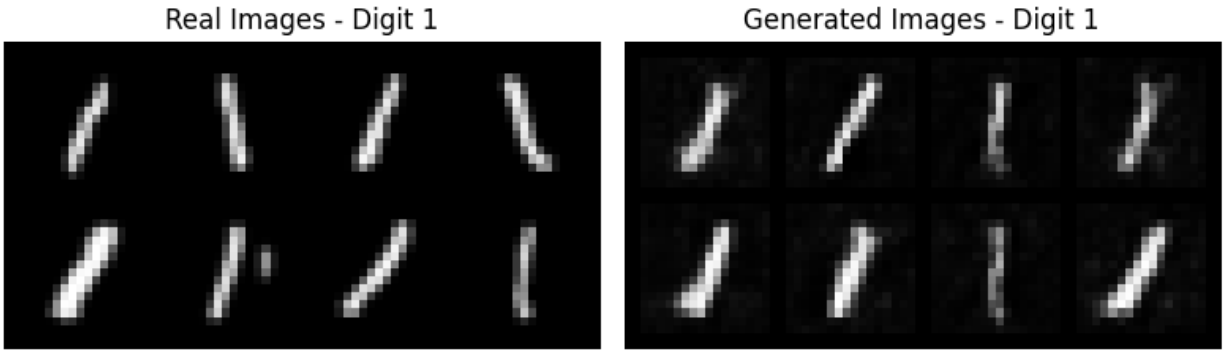
Epoch [15/25] | G Loss: 0.882 | D Loss: 1.224 | Quality: ✓



Epoch [20/25] | G Loss: 0.915 | D Loss: 1.308 | Quality: ✓



Epoch [25/25] | G Loss: 0.921 | D Loss: 1.244 | Quality: ✓



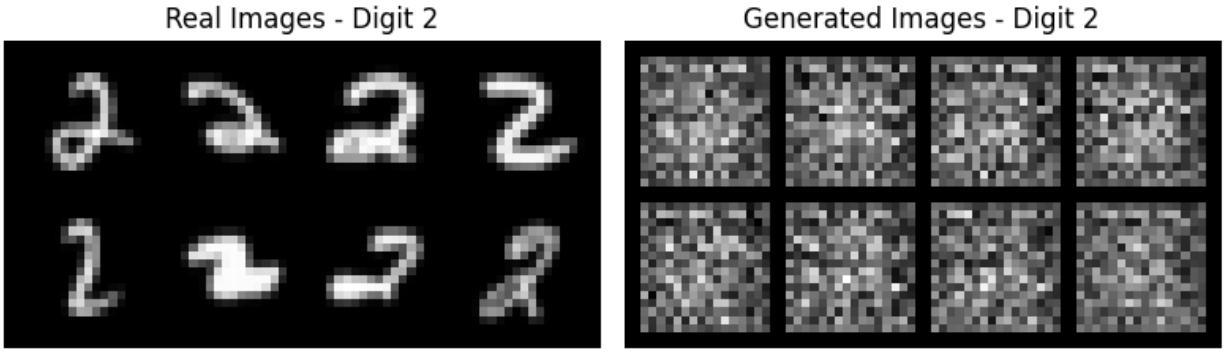
✓ Digit 1 training complete!

==========

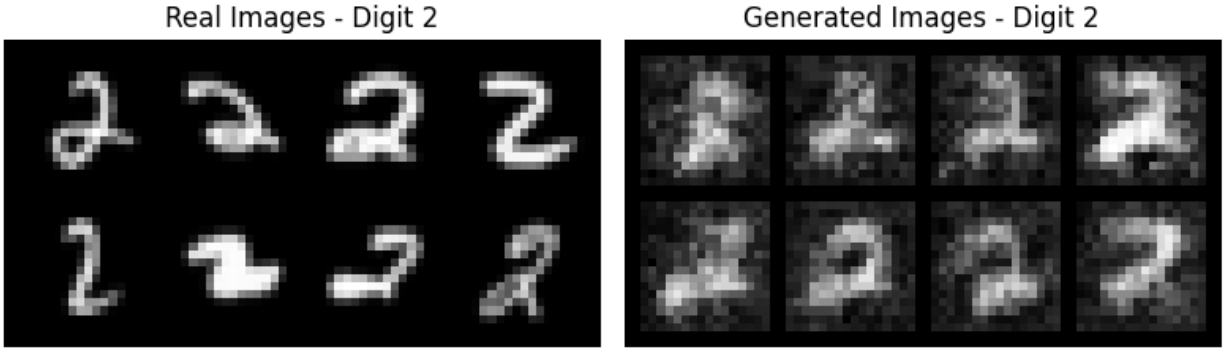
TRAINING DIGIT 2

==========

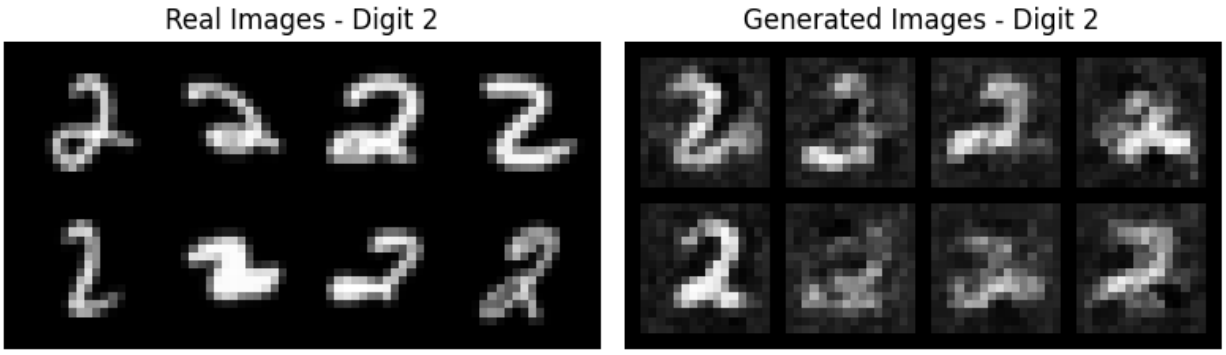
Epoch [1/25] | G Loss: 1.469 | D Loss: 1.368 | Quality: ✅



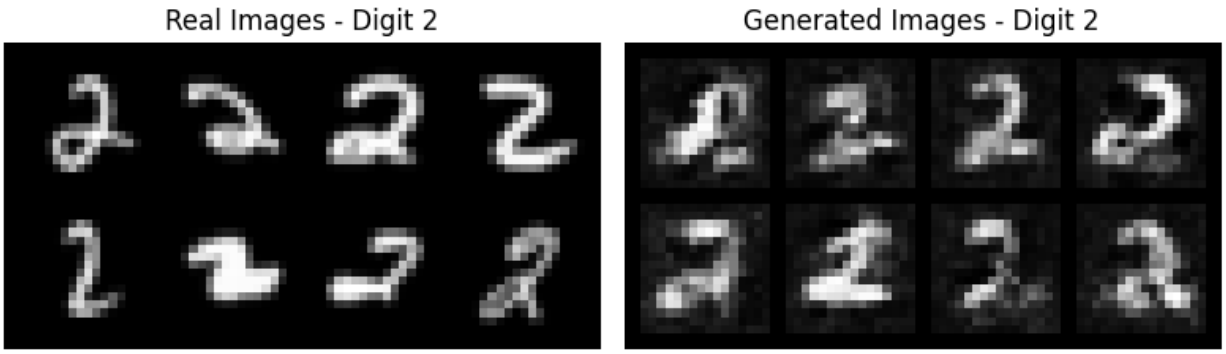
Epoch [5/25] | G Loss: 1.310 | D Loss: 1.153 | Quality: ✅



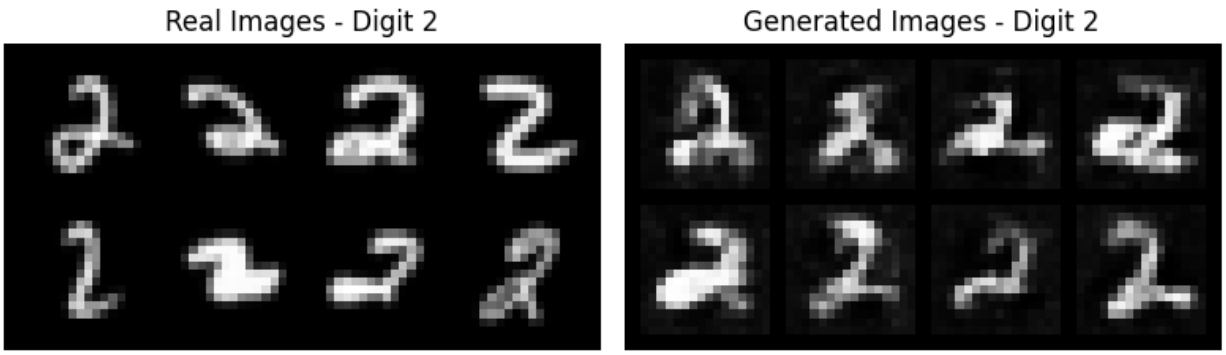
Epoch [10/25] | G Loss: 0.926 | D Loss: 1.209 | Quality: ✅



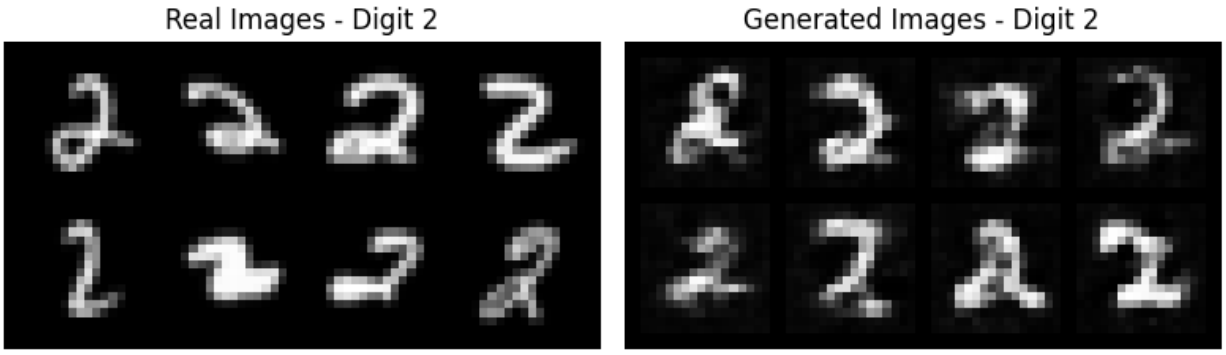
Epoch [15/25] | G Loss: 0.948 | D Loss: 1.236 | Quality: ✓



Epoch [20/25] | G Loss: 0.955 | D Loss: 1.147 | Quality: ✓



Epoch [25/25] | G Loss: 0.974 | D Loss: 1.145 | Quality: ✓



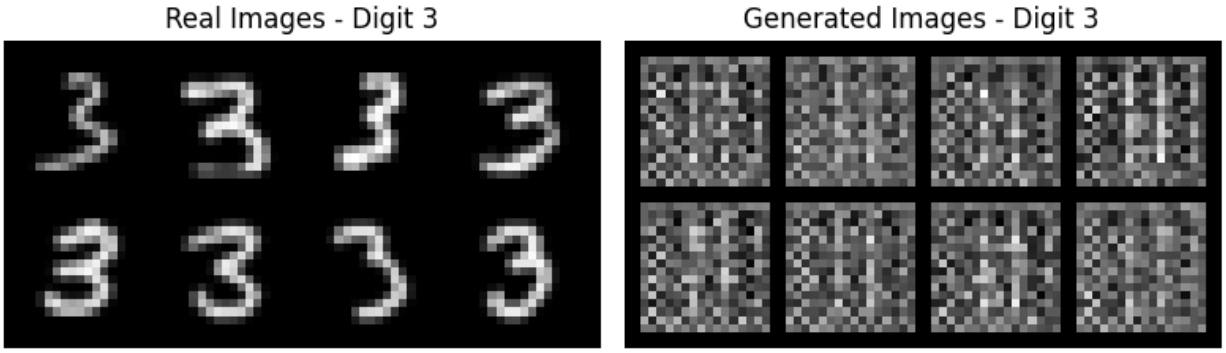
✓ Digit 2 training complete!

==========

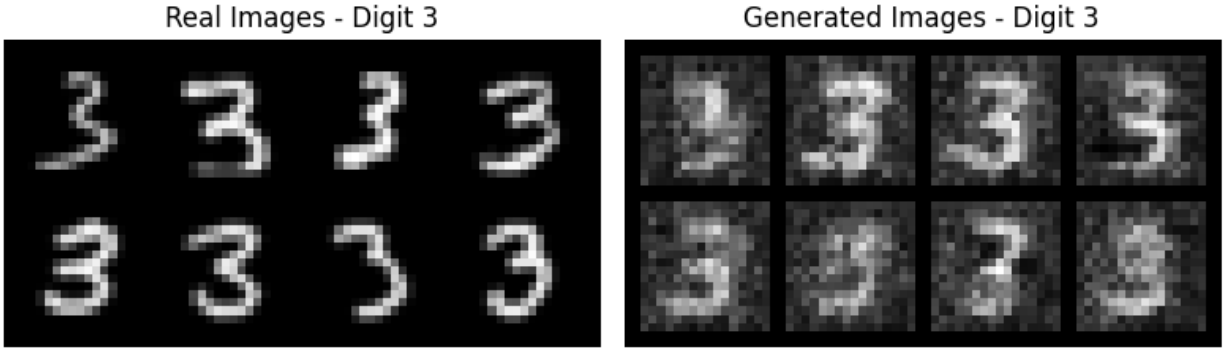
TRAINING DIGIT 3

==========

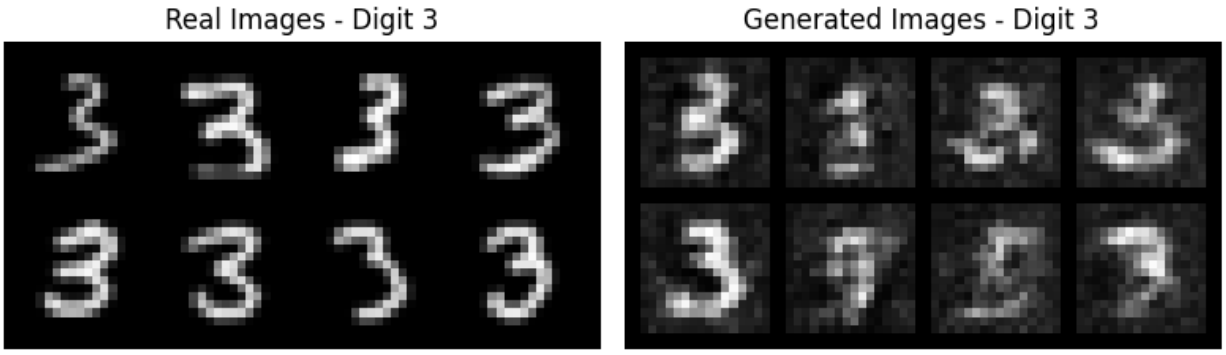
Epoch [1/25] | G Loss: 1.654 | D Loss: 1.299 | Quality: ✓



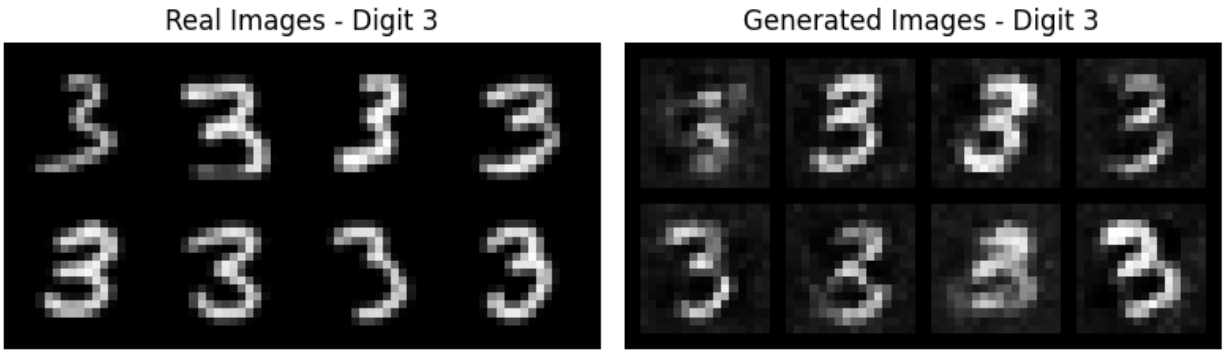
Epoch [5/25] | G Loss: 1.294 | D Loss: 1.093 | Quality: ✓



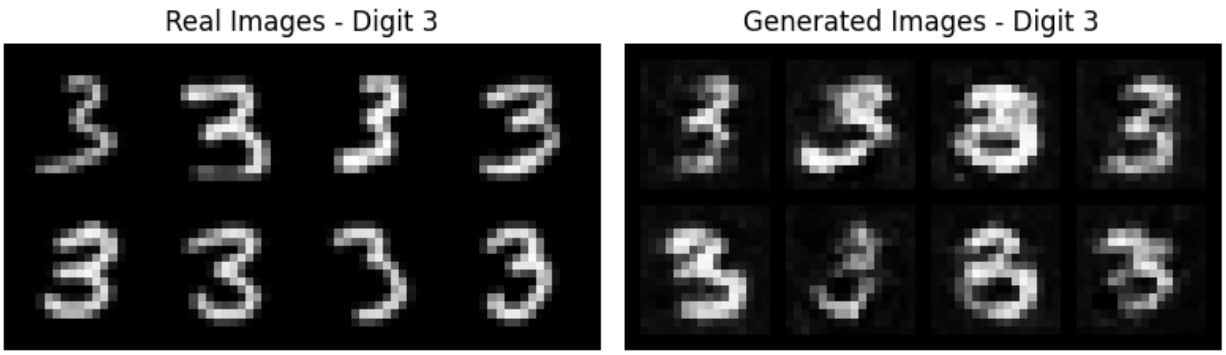
Epoch [10/25] | G Loss: 0.895 | D Loss: 1.202 | Quality: ✓



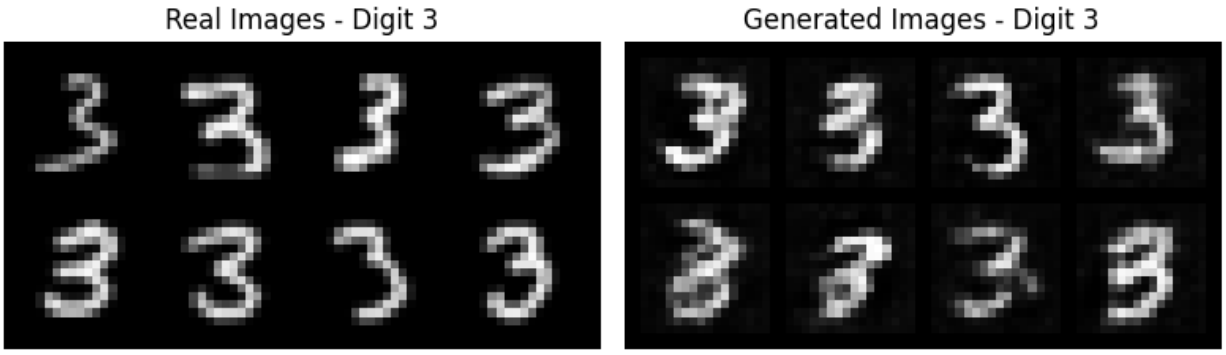
Epoch [15/25] | G Loss: 0.935 | D Loss: 1.203 | Quality: ✓



Epoch [20/25] | G Loss: 0.958 | D Loss: 1.194 | Quality: ✓



Epoch [25/25] | G Loss: 0.957 | D Loss: 1.182 | Quality: ✓



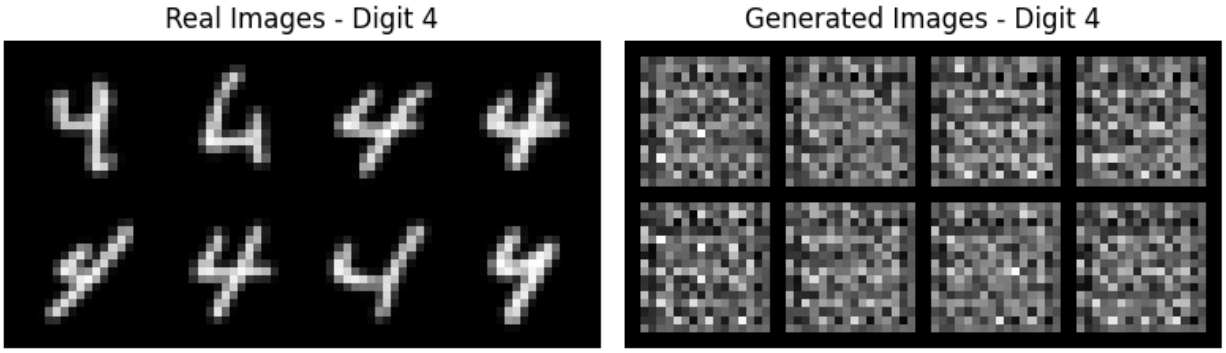
✓ Digit 3 training complete!

==========

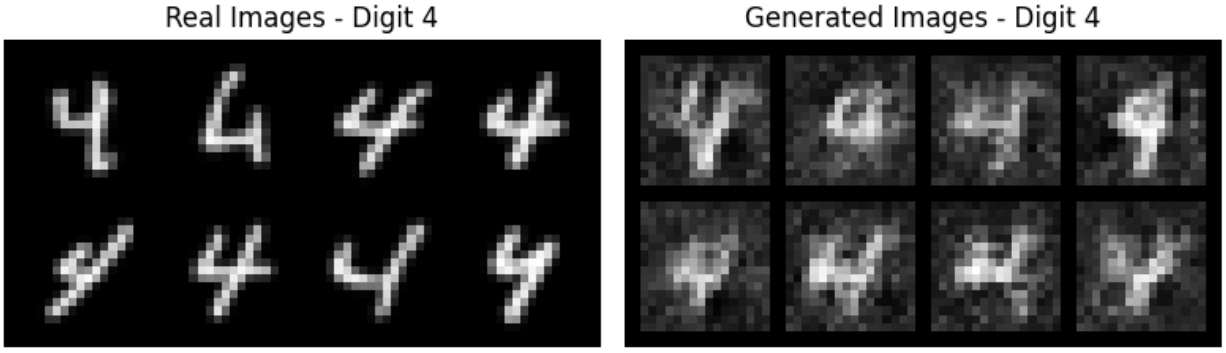
TRAINING DIGIT 4

==========

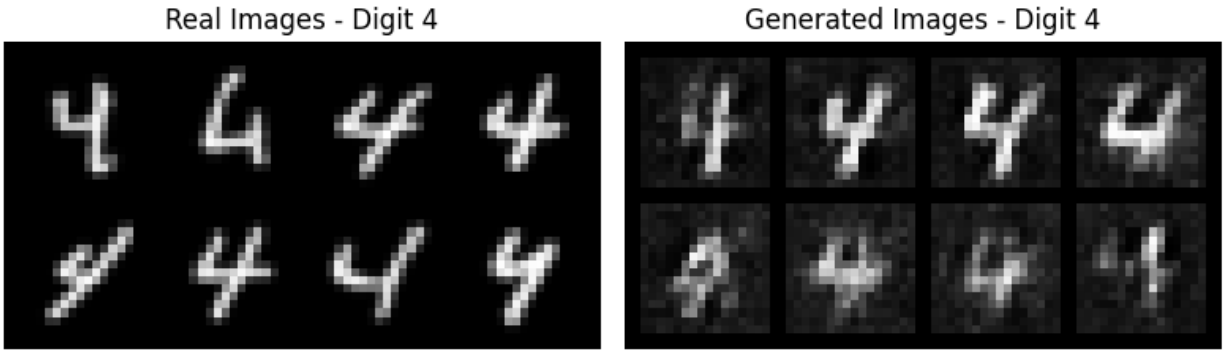
Epoch [1/25] | G Loss: 1.672 | D Loss: 1.268 | Quality: ✓



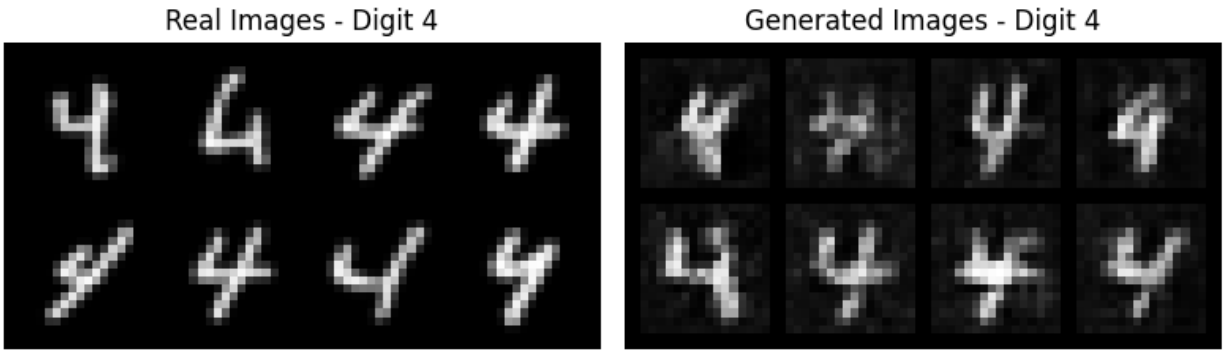
Epoch [5/25] | G Loss: 1.332 | D Loss: 1.146 | Quality: ✓



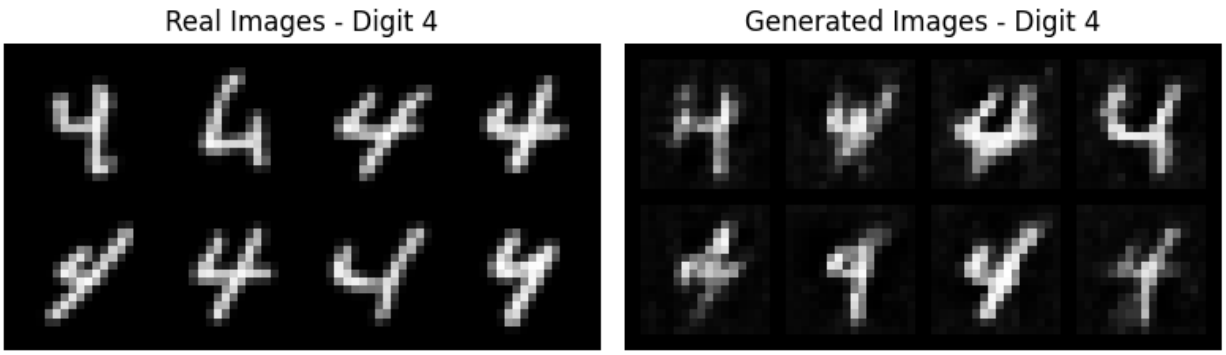
Epoch [10/25] | G Loss: 0.909 | D Loss: 1.301 | Quality: ✓



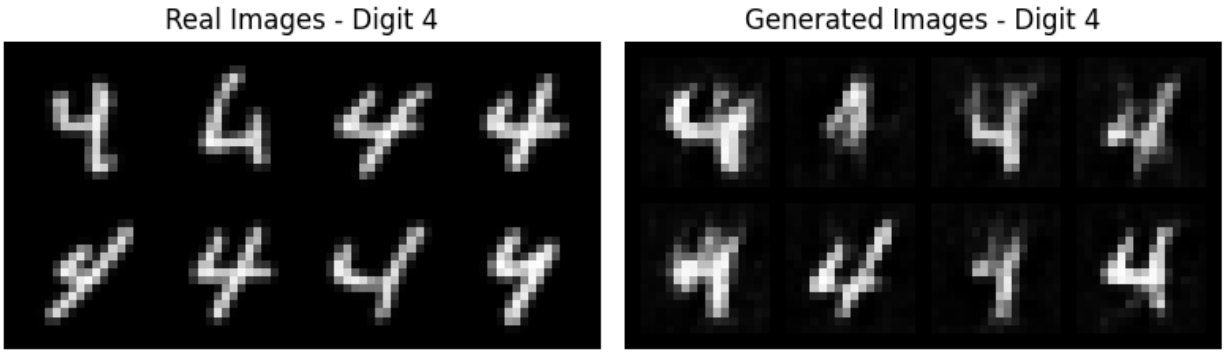
Epoch [15/25] | G Loss: 0.963 | D Loss: 1.243 | Quality: ✓



Epoch [20/25] | G Loss: 0.934 | D Loss: 1.243 | Quality: ✓



Epoch [25/25] | G Loss: 0.956 | D Loss: 1.232 | Quality: ✓



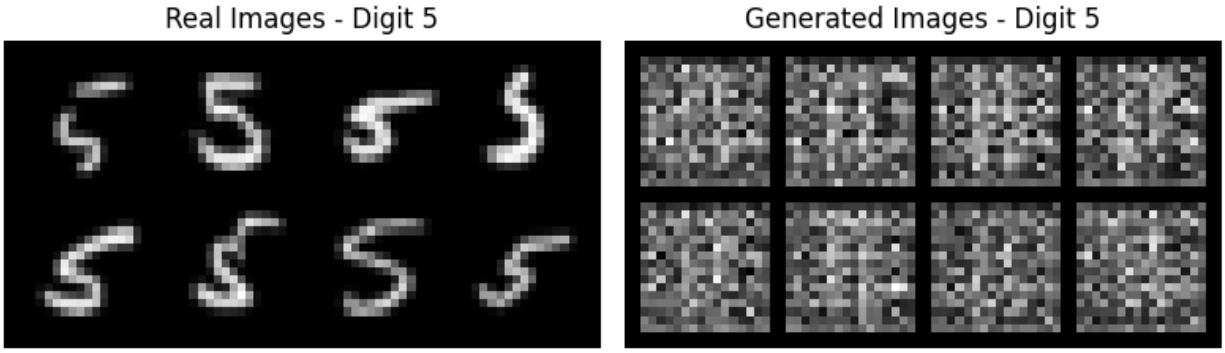
✓ Digit 4 training complete!

==========

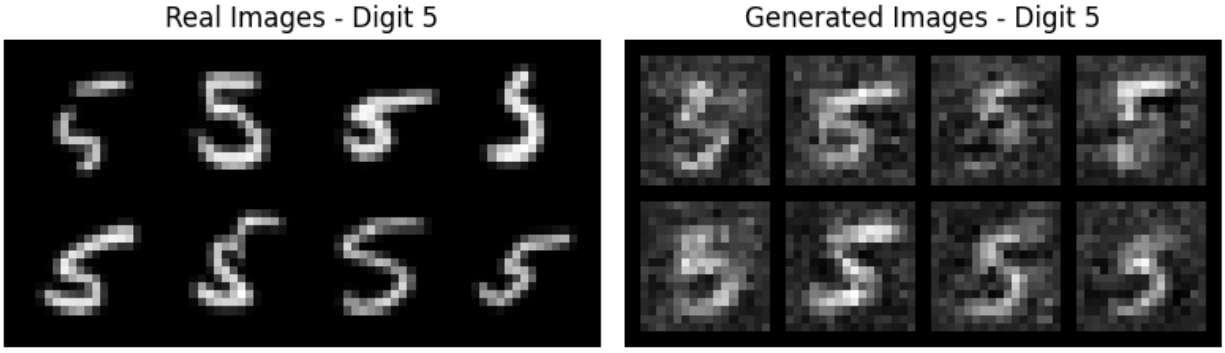
TRAINING DIGIT 5

==========

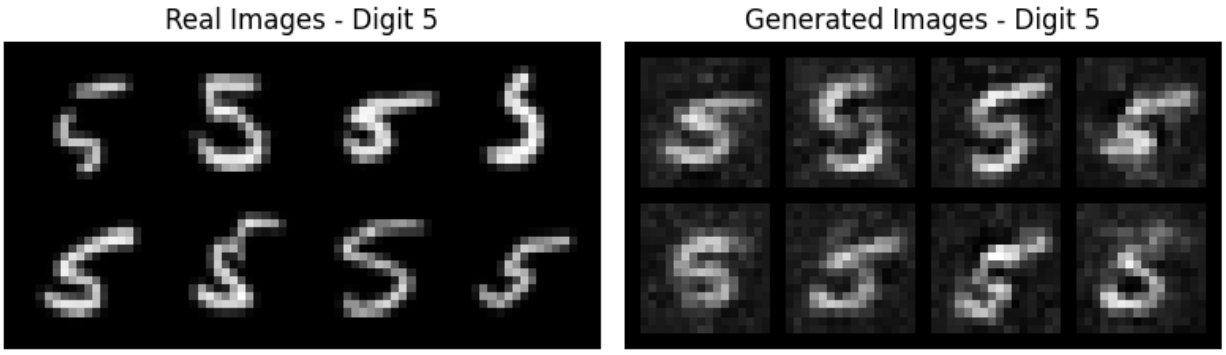
Epoch [1/25] | G Loss: 1.572 | D Loss: 1.261 | Quality: ✓



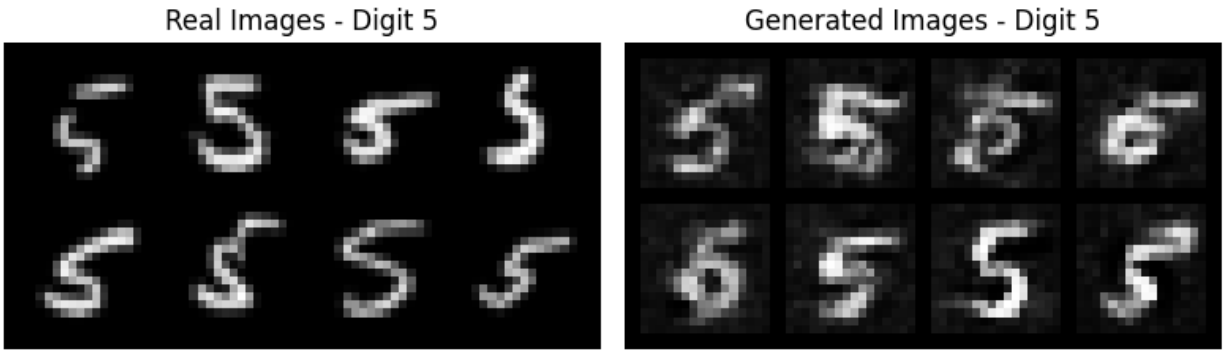
Epoch [5/25] | G Loss: 1.270 | D Loss: 1.104 | Quality: ✓



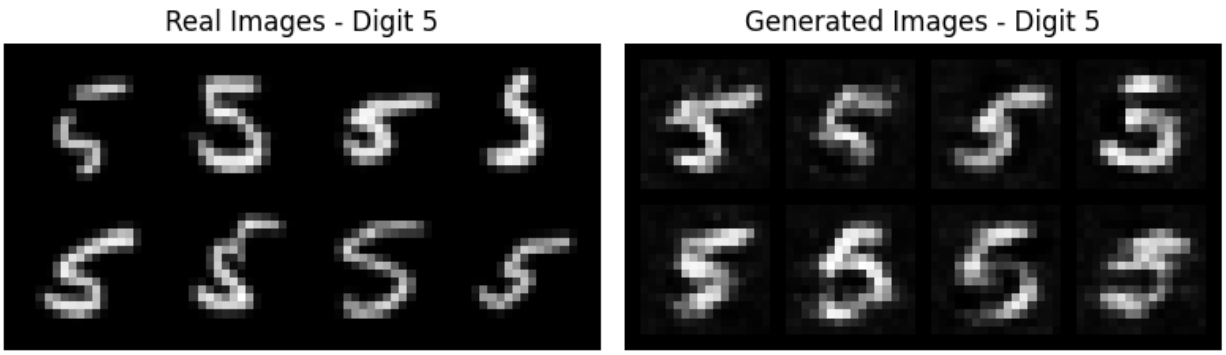
Epoch [10/25] | G Loss: 0.992 | D Loss: 1.202 | Quality: ✓



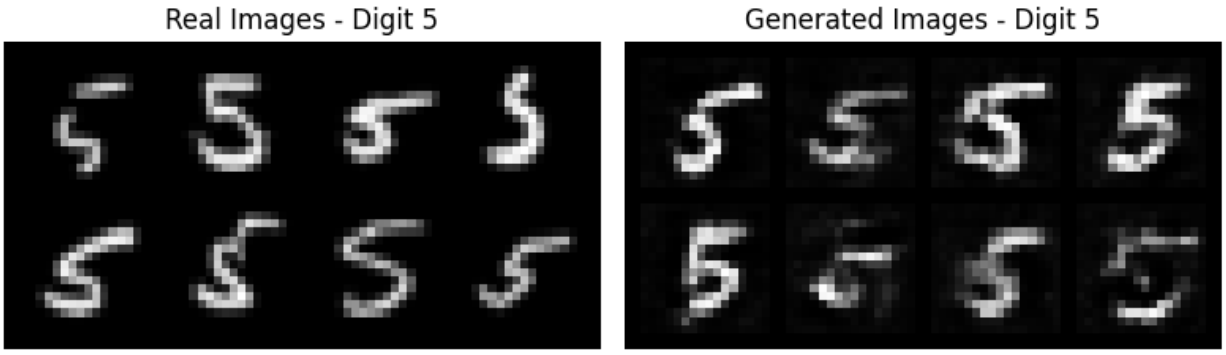
Epoch [15/25] | G Loss: 1.005 | D Loss: 1.202 | Quality: ✓



Epoch [20/25] | G Loss: 0.996 | D Loss: 1.179 | Quality: ✓



Epoch [25/25] | G Loss: 1.011 | D Loss: 1.126 | Quality: ✓



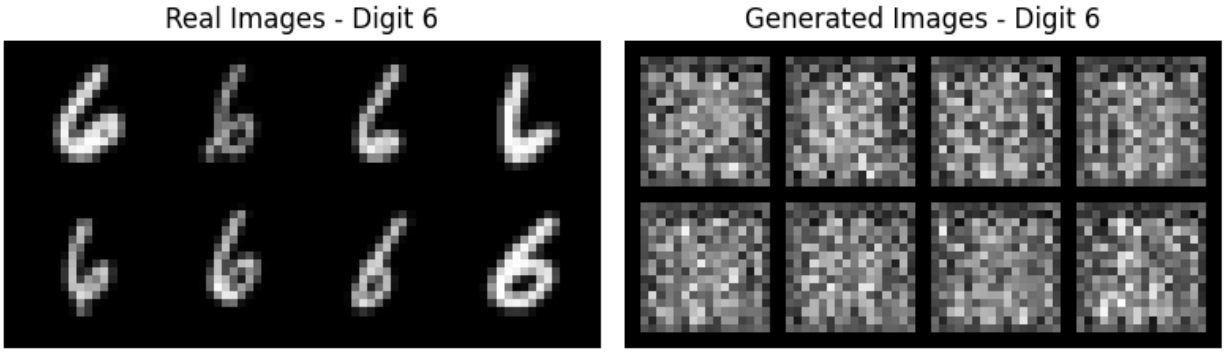
✓ Digit 5 training complete!

==========

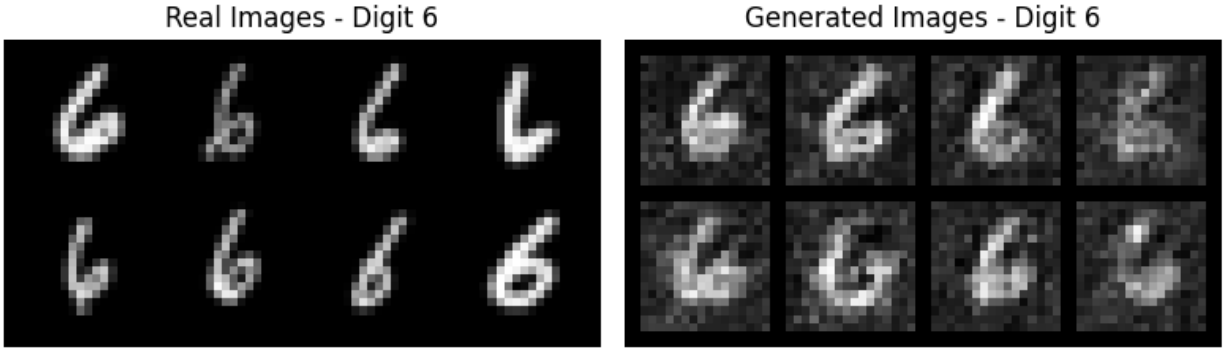
TRAINING DIGIT 6

==========

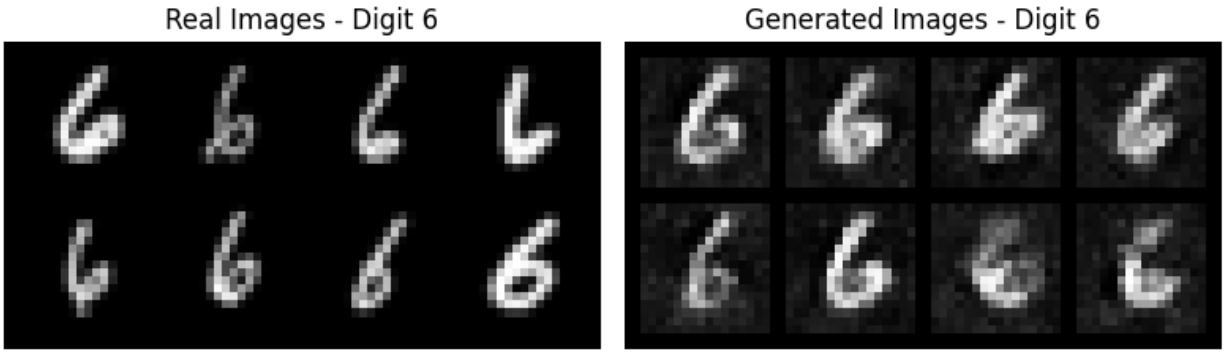
Epoch [1/25] | G Loss: 1.611 | D Loss: 1.320 | Quality: ✓



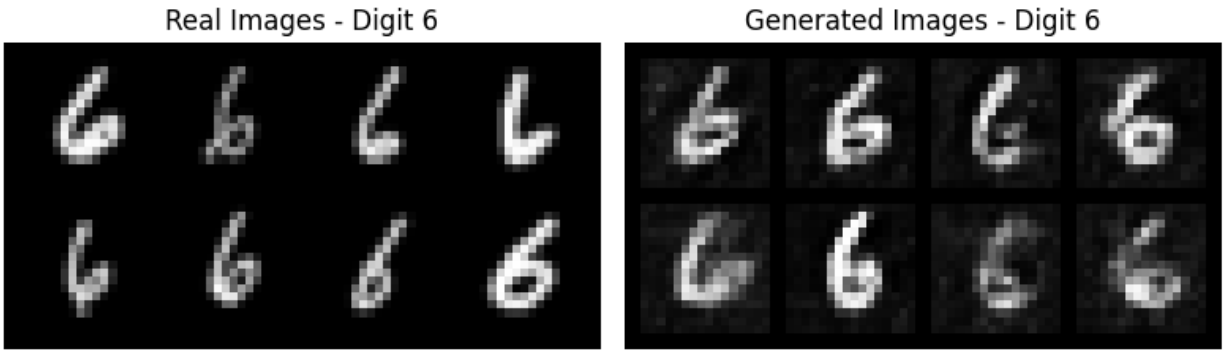
Epoch [5/25] | G Loss: 1.286 | D Loss: 1.112 | Quality: ✓



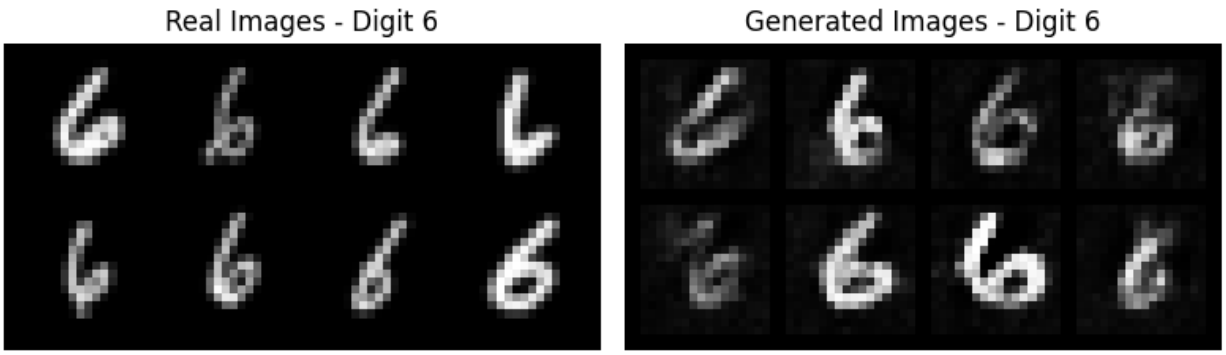
Epoch [10/25] | G Loss: 0.946 | D Loss: 1.169 | Quality: ✓



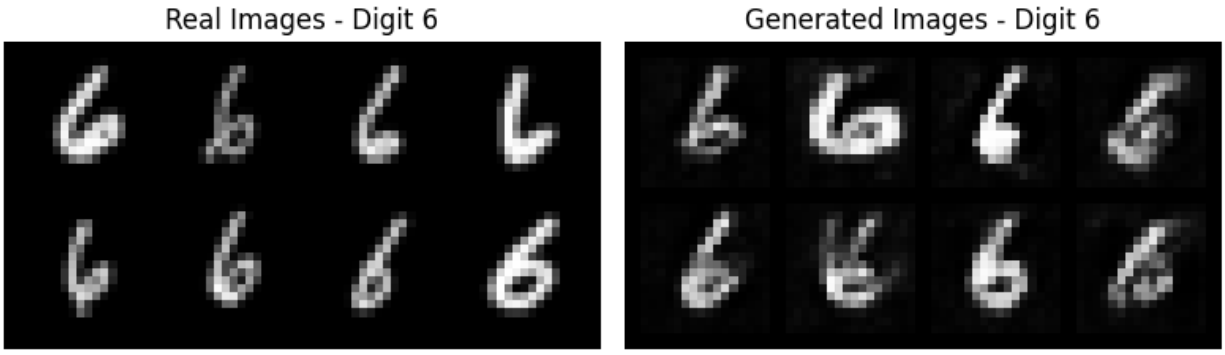
Epoch [15/25] | G Loss: 0.950 | D Loss: 1.181 | Quality: ✓



Epoch [20/25] | G Loss: 0.977 | D Loss: 1.208 | Quality: ✓



Epoch [25/25] | G Loss: 0.971 | D Loss: 1.199 | Quality: ✓



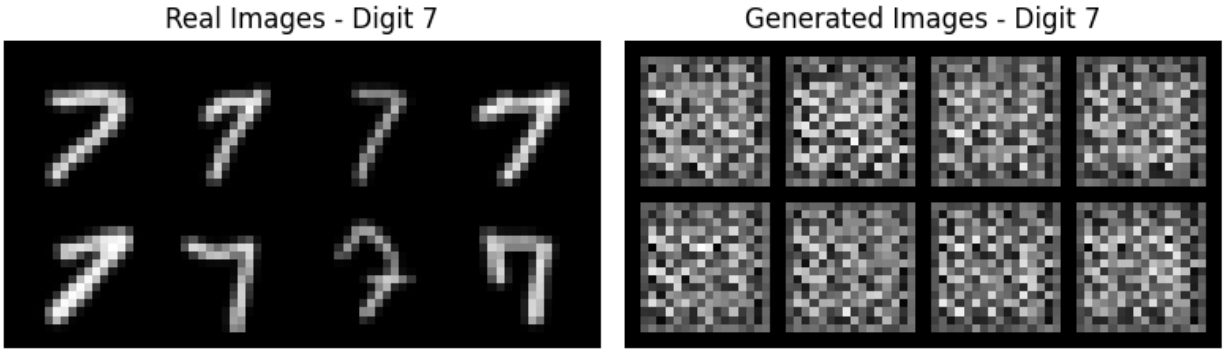
✓ Digit 6 training complete!

==========

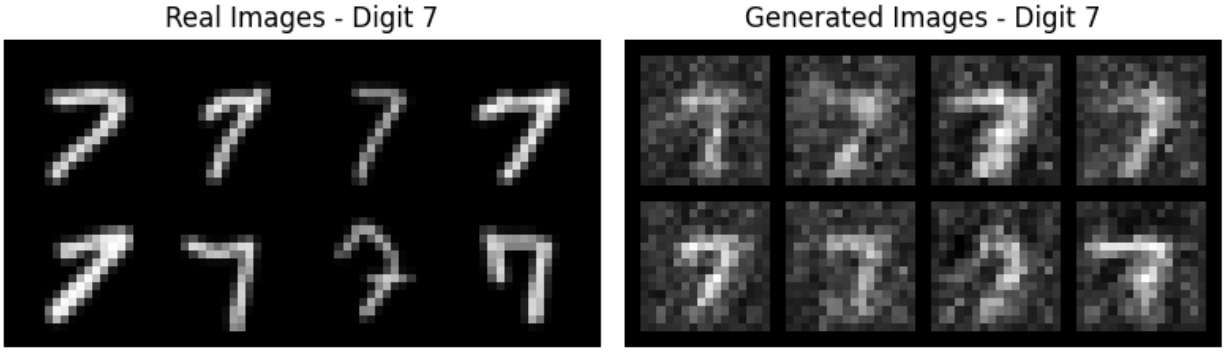
TRAINING DIGIT 7

==========

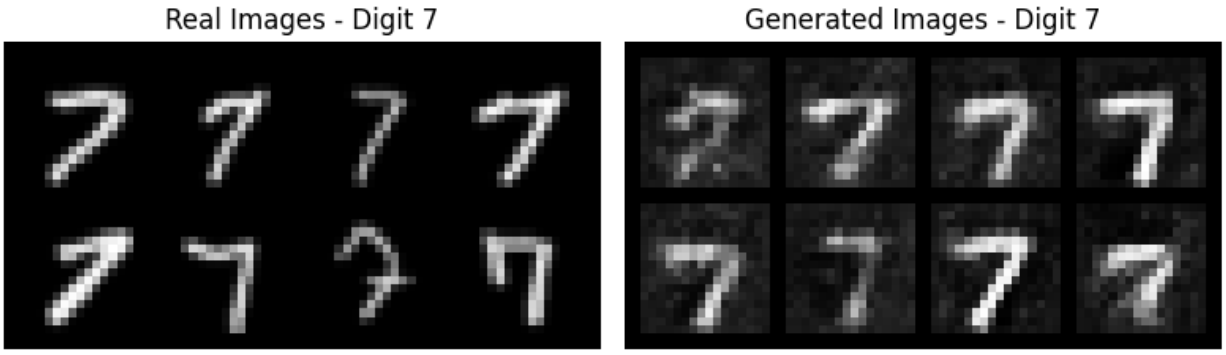
Epoch [1/25] | G Loss: 1.965 | D Loss: 1.153 | Quality: ✓



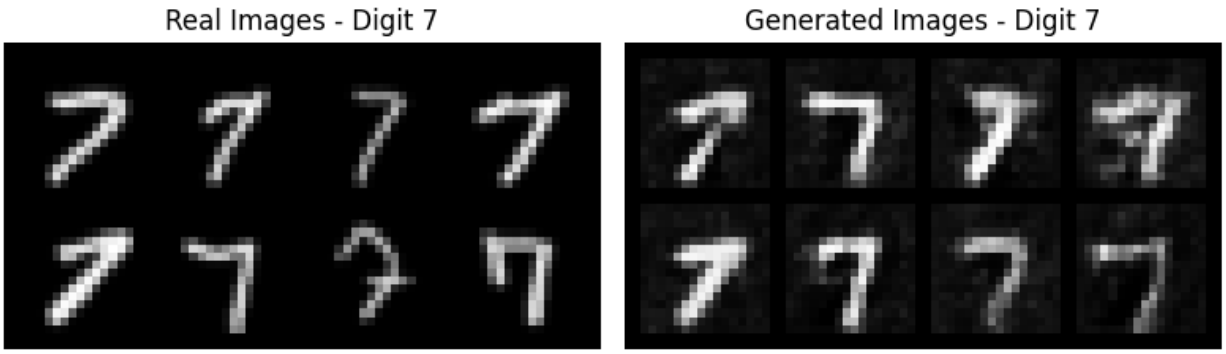
Epoch [5/25] | G Loss: 1.259 | D Loss: 1.035 | Quality: ✓



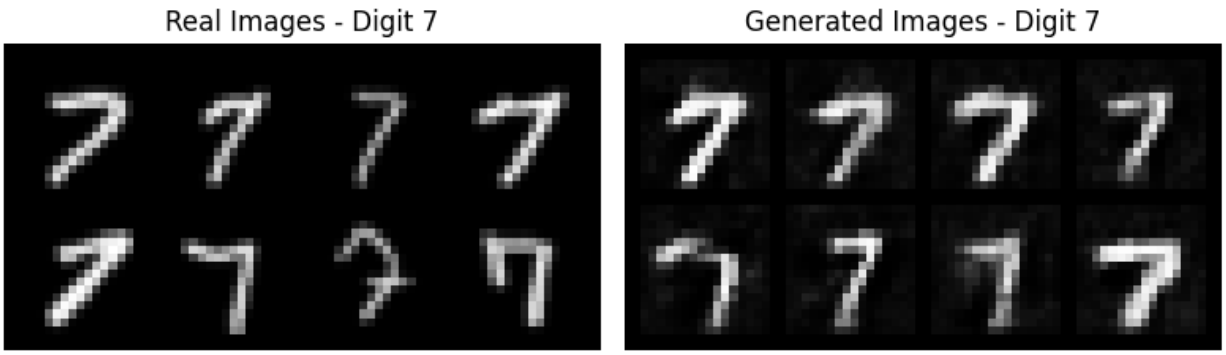
Epoch [10/25] | G Loss: 1.018 | D Loss: 1.237 | Quality: ✓



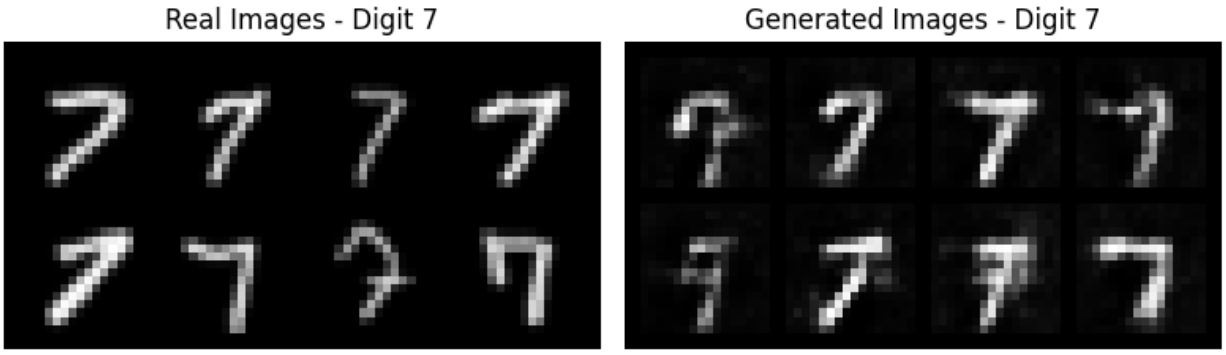
Epoch [15/25] | G Loss: 0.928 | D Loss: 1.236 | Quality: ✓



Epoch [20/25] | G Loss: 0.912 | D Loss: 1.192 | Quality: ✓



Epoch [25/25] | G Loss: 0.951 | D Loss: 1.226 | Quality: ✓



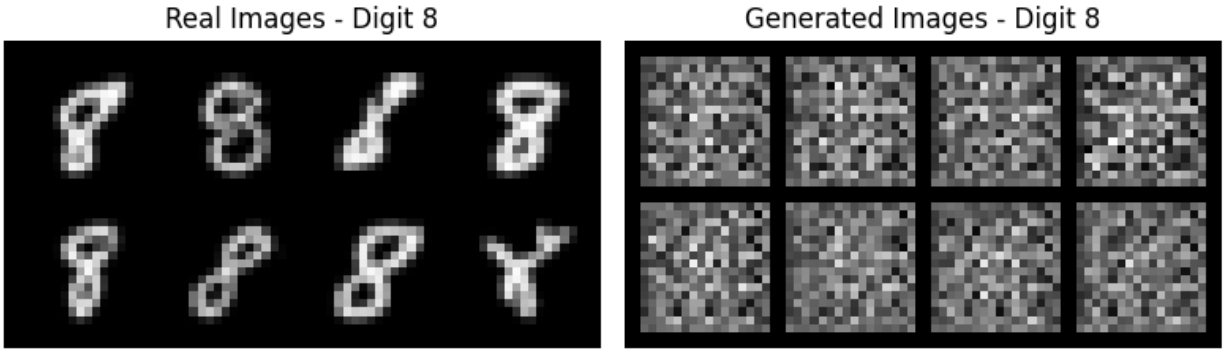
✓ Digit 7 training complete!

==========

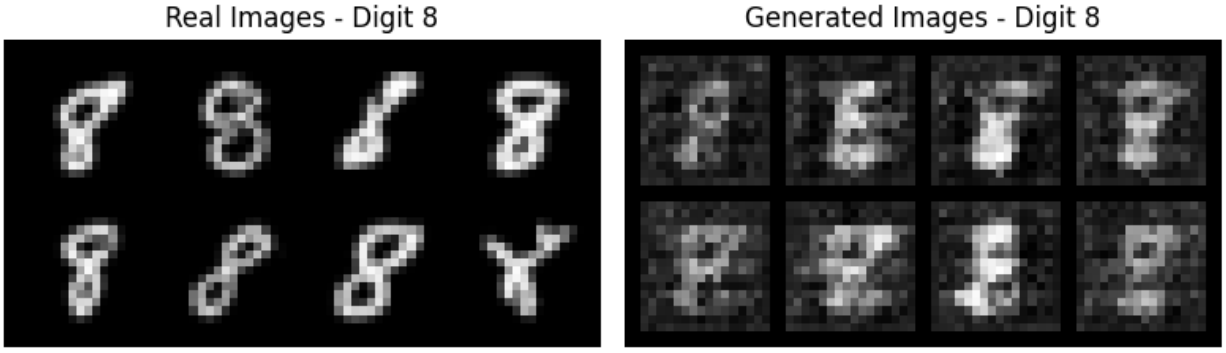
TRAINING DIGIT 8

==========

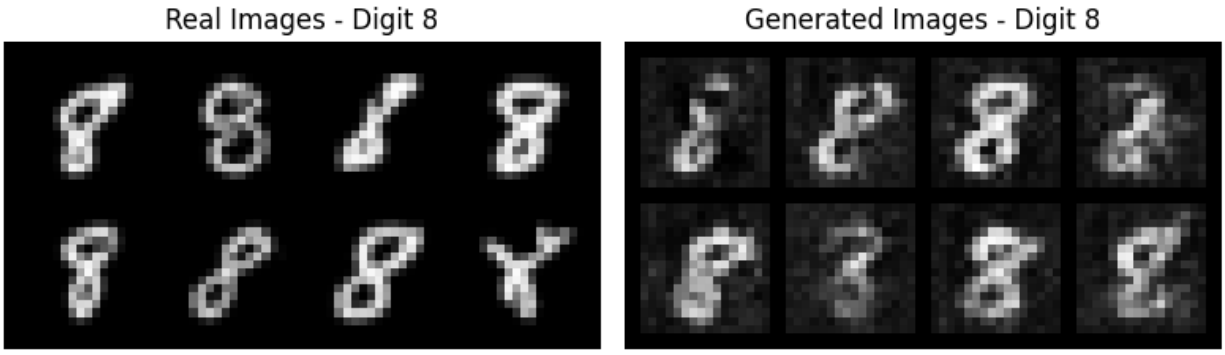
Epoch [1/25] | G Loss: 1.677 | D Loss: 1.221 | Quality: ✓



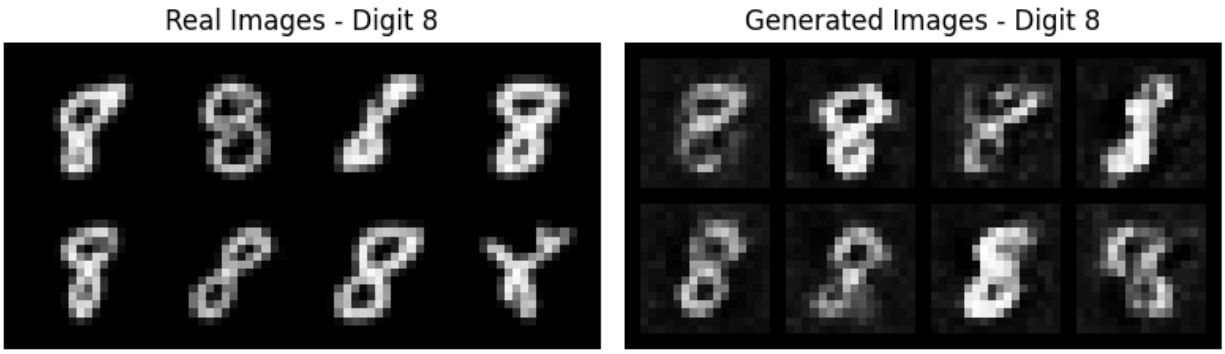
Epoch [5/25] | G Loss: 1.428 | D Loss: 1.112 | Quality: ✓



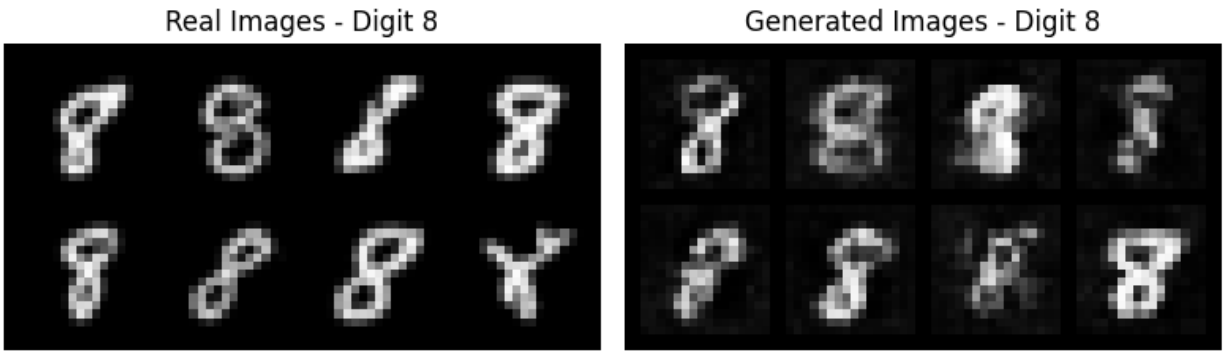
Epoch [10/25] | G Loss: 0.889 | D Loss: 1.226 | Quality: ✓



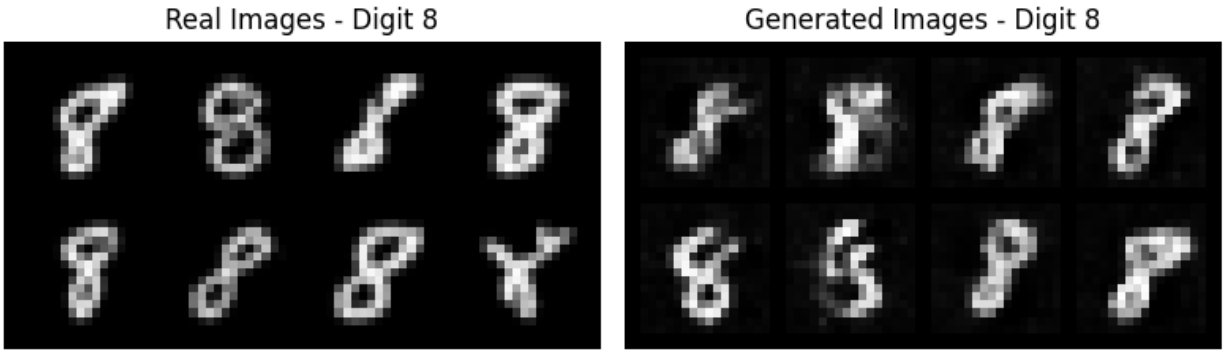
Epoch [15/25] | G Loss: 0.921 | D Loss: 1.211 | Quality: ✓



Epoch [20/25] | G Loss: 0.961 | D Loss: 1.212 | Quality: ✓



Epoch [25/25] | G Loss: 0.968 | D Loss: 1.225 | Quality: ✓



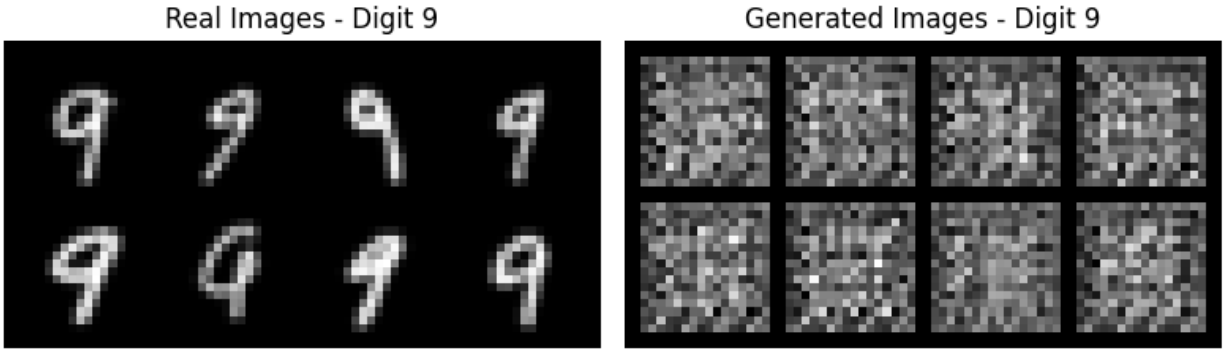
✓ Digit 8 training complete!

==========

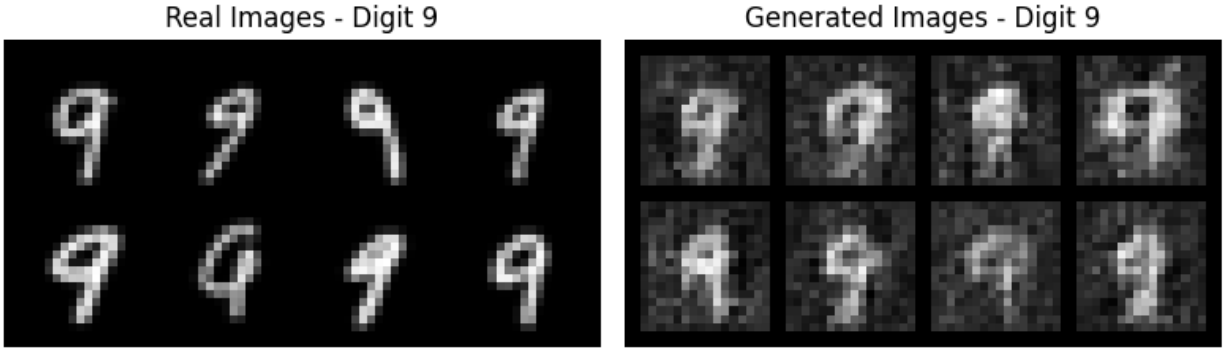
TRAINING DIGIT 9

==========

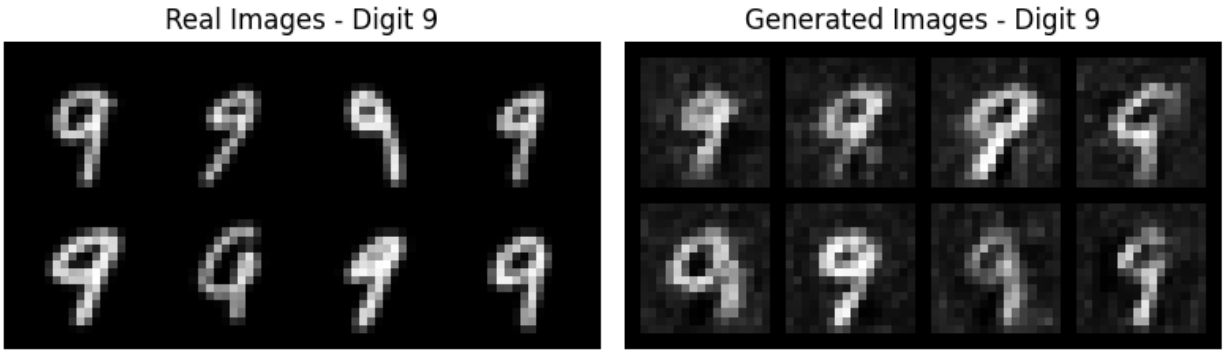
Epoch [1/25] | G Loss: 1.598 | D Loss: 1.172 | Quality: ✓



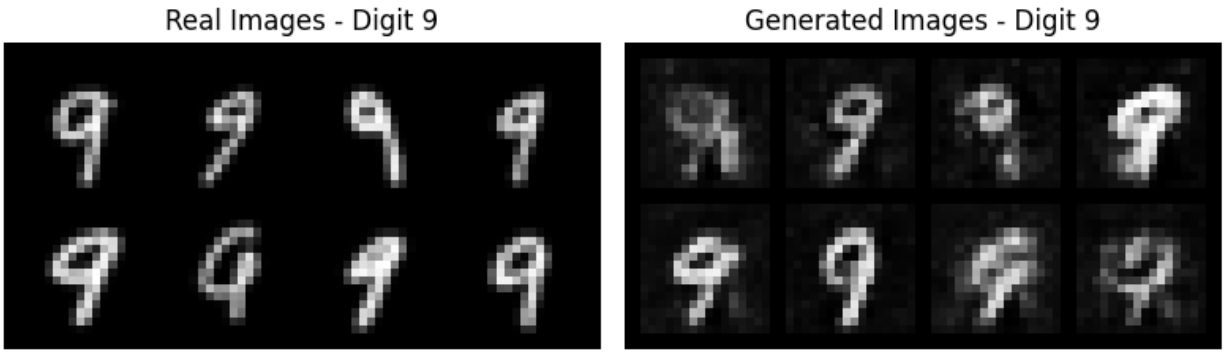
Epoch [5/25] | G Loss: 1.406 | D Loss: 1.080 | Quality: ✓



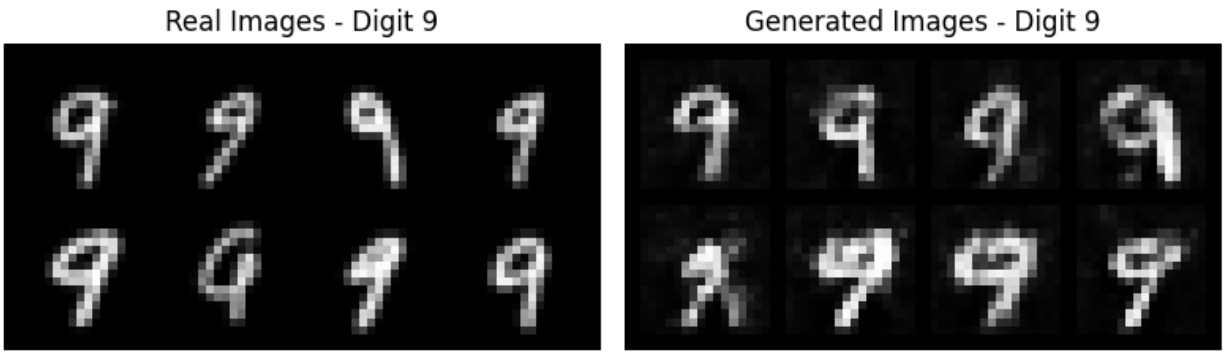
Epoch [10/25] | G Loss: 0.972 | D Loss: 1.233 | Quality: ✓



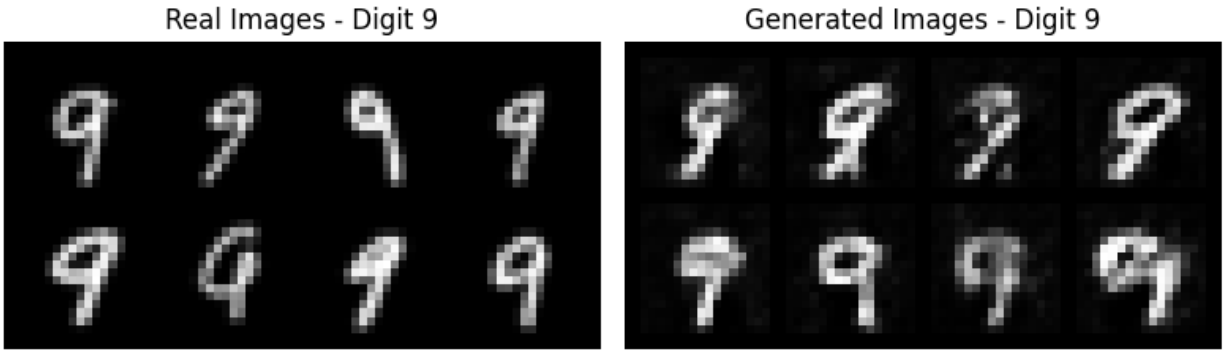
Epoch [15/25] | G Loss: 0.943 | D Loss: 1.212 | Quality: ✓



Epoch [20/25] | G Loss: 0.942 | D Loss: 1.202 | Quality: ✓



Epoch [25/25] | G Loss: 0.977 | D Loss: 1.266 | Quality: ✓



✓ Digit 9 training complete!

ALL DIGITS TRAINING COMPLETE!

Block 13: Generate All Trained Digits Grid (6 samples per digit)

INPUT

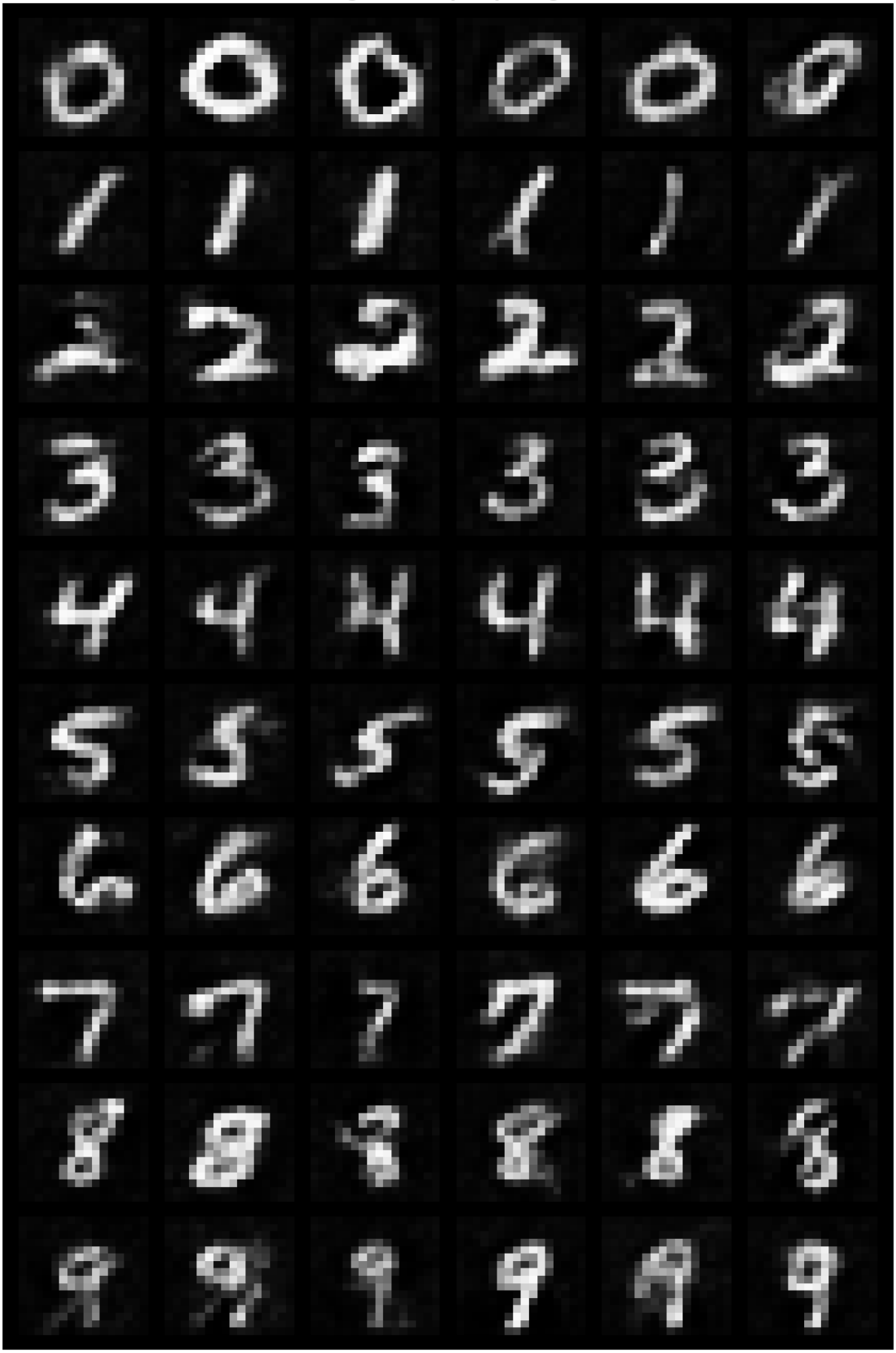
```
# Load all trained generators and show all digits together

print("Loading all trained generators...\n")

generator = Generator().to(device)

all_samples = []

for digit in range(10):

        generator.load_state_dict(trained_generators[digit])

        generator.eval()

        with torch.no_grad():

                noise = torch.randn(6, latent_dim, 1, 1, device=device)

                samples = generator(noise)

                all_samples.append(samples)

final_grid = make_grid(torch.cat(all_samples), nrow=6, normalize=True)

plt.figure(figsize=(12, 18))

plt.imshow(final_grid.cpu().permute(1, 2, 0))
```

```
plt.title("All Trained Digits (6 samples per digit, rows 0-9)", fontsize=14, fontweight='bold')

plt.axis("off")

plt.tight_layout()

plt.show()
```

OUTPUT:

Each row shows 6 samples of digits 0-9

All Trained Digits (6 samples per digit, rows 0-9)



✓ Generated 60 images (6 per digit) showing all trained digits 0-9

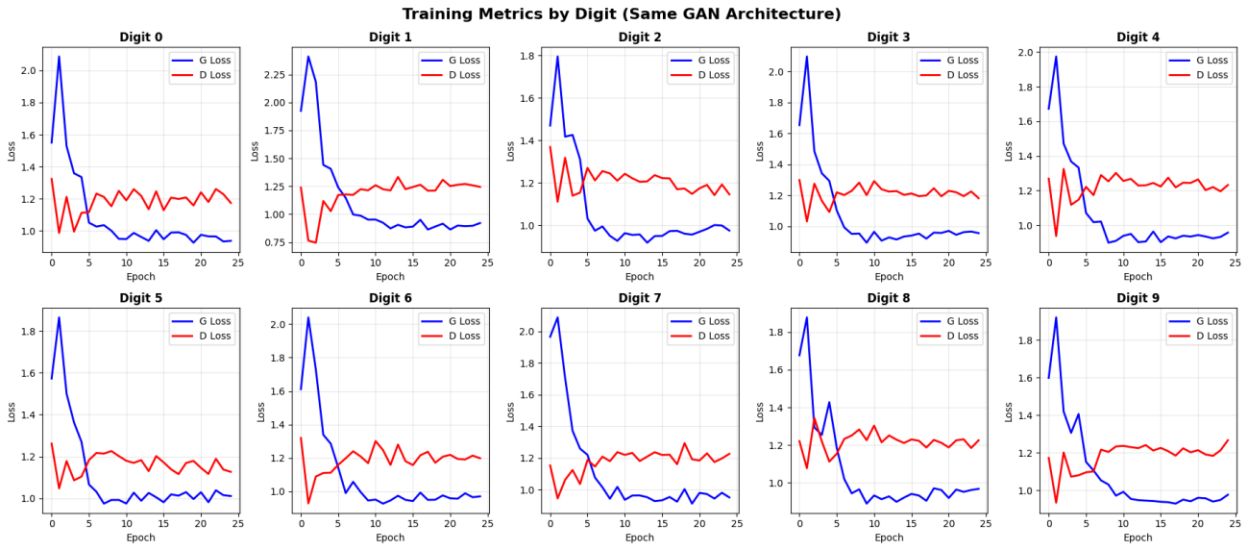
Block 14: Plot Training Metrics by Digit

INPUT

```
# Plot training metrics for all digits
fig, axes = plt.subplots(2, 5, figsize=(18, 8))
fig.suptitle('Training Metrics by Digit (Same GAN Architecture)', fontsize=16, fontweight='bold')
for digit in range(10):
    row = digit // 5
    col = digit % 5
    ax = axes[row, col]
    ax.plot(metrics_history[digit]['g_loss'], label='G Loss', color='blue', linewidth=2)
    ax.plot(metrics_history[digit]['d_loss'], label='D Loss', color='red', linewidth=2)
    ax.set_title(f'Digit {digit}', fontweight='bold')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.legend(loc='upper right')
    ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

OUTPUT:

Training Metrics by Digit (Same GAN Architecture)



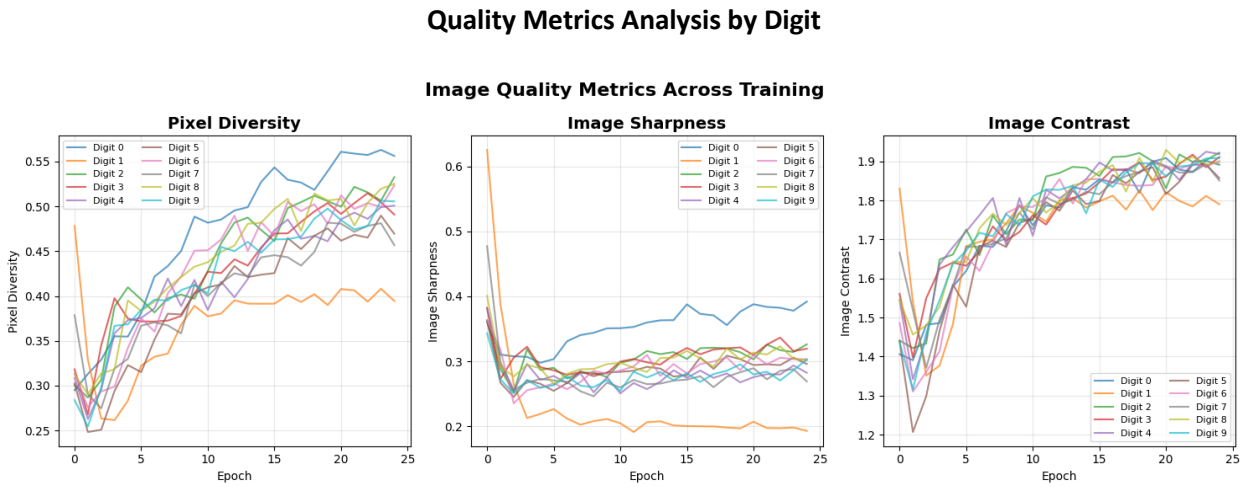
Block 15: Plot Image Quality Metrics

INPUT

```
# Image quality metrics visualization

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

metrics_names = ['diversity', 'sharpness', 'contrast']

titles = ['Pixel Diversity', 'Image Sharpness', 'Image Contrast']

for idx, (metric_name, title) in enumerate(zip(metrics_names, titles)):
        ax = axes[idx]
        for digit in range(10):
                values = [q[metric_name] for q in metrics_history[digit]['quality']]
                ax.plot(values, label=f'Digit {digit}', alpha=0.7, linewidth=1.5)
        ax.set_title(title, fontsize=14, fontweight='bold')
        ax.set_xlabel('Epoch')
        ax.set_ylabel(title)
        ax.legend(loc='best', ncol=2, fontsize=8)
```

```
    ax.grid(True, alpha=0.3)

plt.suptitle('Image Quality Metrics Across Training', fontsize=16, fontweight='bold')

plt.tight_layout()

plt.show()
```

OUTPUT:



Block 16: Display Final Training Summary

INPUT

```
# Final quality summary

print("\n" + "="*70)

print("FINAL TRAINING SUMMARY")

print("="*70)

for digit in range(10):

    final_g_loss = metrics_history[digit]['g_loss'][-1]

    final_d_loss = metrics_history[digit]['d_loss'][-1]

    final_quality = metrics_history[digit]['quality'][-1]

    if final_g_loss < 2.5:
```

```
            status = "Excellent"
        elif final_g_loss < 4.5:
            status = "Good"
        else:
            status = "Needs improvement"
        print(f"Digit {digit}: G Loss: {final_g_loss:.3f} | D Loss: {final_d_loss:.3f} | {status}")
        print(f" Quality - Diversity: {final_quality['diversity']:.4f} | "f"Sharpness:
        {final_quality['sharpness']:.4f} | "f"Contrast: {final_quality['contrast']:.4f}")
print("="*70)
```

OUTPUT:

```
======================================================================
FINAL TRAINING SUMMARY
======================================================================
Digit 0: G Loss: 0.938 | D Loss: 1.174 | Excellent
Quality - Diversity: 0.3618 | Sharpness: 0.3180 | Contrast: 1.8499
Digit 1: G Loss: 0.921 | D Loss: 1.244 | Excellent
Quality - Diversity: 0.4215 | Sharpness: 0.3151 | Contrast: 1.8377
Digit 2: G Loss: 0.974 | D Loss: 1.145 | Excellent
Quality - Diversity: 0.3846 | Sharpness: 0.3184 | Contrast: 1.8270
Digit 3: G Loss: 0.957 | D Loss: 1.182 | Excellent
Quality - Diversity: 0.3766 | Sharpness: 0.3546 | Contrast: 1.8429
Digit 4: G Loss: 0.956 | D Loss: 1.232 | Excellent
Quality - Diversity: 0.2945 | Sharpness: 0.3405 | Contrast: 1.8848
Digit 5: G Loss: 1.011 | D Loss: 1.126 | Excellent
Quality - Diversity: 0.4365 | Sharpness: 0.3555 | Contrast: 1.8409
```

Digit 6: G Loss: 0.971 | D Loss: 1.199 | Excellent

Quality - Diversity: 0.3759 | Sharpness: 0.3711 | Contrast: 1.8187

Digit 7: G Loss: 0.951 | D Loss: 1.226 | Excellent

Quality - Diversity: 0.2333 | Sharpness: 0.3866 | Contrast: 1.8055

Digit 8: G Loss: 0.968 | D Loss: 1.225 | Excellent

Quality - Diversity: 0.3930 | Sharpness: 0.3930 | Contrast: 1.8020

Digit 9: G Loss: 0.977 | D Loss: 1.266 | Excellent

Quality - Diversity: 0.3461 | Sharpness: 0.3774 | Contrast: 1.8815

==========

Block 17: Interactive Digit Generation

INPUT

```
# Interactive generation function
def generate_digit(digit, num_samples=5):
    """
    Generate images for a specific digit using the trained generator.
    Args:
        digit (int): Digit to generate (0-9)
        num_samples (int): Number of samples to generate
    """
    if digit < 0 or digit > 9:
        print("Error: Please enter a digit between 0 and 9")
        return
    print(f"Generating {num_samples} images for digit {digit}...")
```

```
        # Load trained generator for this digit
        generator = Generator().to(device)
        generator.load_state_dict(trained_generators[digit])
        generator.eval()
        with torch.no_grad():
            noise = torch.randn(num_samples, latent_dim, 1, 1, device=device)
            generated_images = generator(noise)
        # Display
        grid = make_grid(generated_images, nrow=num_samples, normalize=True)
        plt.figure(figsize=(num_samples * 2, 2))
        plt.imshow(grid.cpu().permute(1, 2, 0))
        plt.title(f"Generated Images for Digit: {digit}", fontsize=14, fontweight='bold')
        plt.axis("off")
        plt.tight_layout()
        plt.show()
        # Quality metrics
        quality = calculate_image_quality(generated_images)
        print(f"\nQuality Metrics:")
        print(f" Diversity: {quality['diversity']:.4f}")
        print(f" Sharpness: {quality['sharpness']:.4f}")
        print(f" Contrast: {quality['contrast']:.4f}")
print("\n" + "="*70)
print("INTERACTIVE GENERATION READY")
print("="*70)
```

```
print("Usage: generate_digit(digit, num_samples=5)")

print("Example: generate_digit(3, 8) # Generate 8 images of digit 3")

print("="*70)
```

OUTPUT:

```
======================================================================

INTERACTIVE GENERATION READY

======================================================================

Usage: generate_digit(digit, num_samples=5)

Example: generate_digit(3, 8) # Generate 8 images of digit 3

======================================================================

Try it yourself:

Select Digit: Digit 0 Generate (5 samples)

Generating 5 images of digit 0...
```